

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Cougar: Cubic Root Verifier Inner Product Argument under Discrete Logarithm Assumption

HYEONBUM LEE¹, SEUNGHUN PAIK^{2,3}, HYUNJUNG SON^{2,3}, and JAE HONG SEO^{2,3}

¹BK21 FOUR Intelligence Computing, Seoul National University, Seoul 08826, Republic of Korea

²Department of Mathematics, Hanyang University, Seoul 04763, Republic of Korea

³Research Institute for Natural Sciences, Hanyang University, Seoul 04763, Republic of Korea

Corresponding author: Jae Hong Seo (e-mail: jaehongseo@hanyang.ac.kr)

This research was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP), grant funded by the Korea Government (MSIT) (RS-2021-II210727, 90%), in part by Culture, Sports and Tourism R&D Program through the Korea Creative Content Agency grant funded by the Ministry of Culture, Sports and Tourism (RS-2024-00332210, 10%)

ABSTRACT An inner product argument (IPA) is a cryptographic proof system that serves as a fundamental building block for various applications, such as zero knowledge proofs and verifiable computation. Bulletproofs (IEEE S&P 2018), a well-known IPA under the discrete logarithm (DL) assumption, features a short, logarithmically-sized proof, making it suitable for blockchain applications. However, its major drawback is the linear verifier cost ($O(N)$), which presents a significant bottleneck in settings like verifiable computation. To address this, recent advancements have successfully reduced the verification complexity to square-root order ($O(\sqrt{N})$) under the same assumption (e.g., Asiacrypt 2022, IEEE TIFS).

In this work, we propose **Cougar**, a novel IPA that breaks this square-root barrier to achieve an unprecedented **cubic-root verifier complexity** ($O(\sqrt[3]{N})$), while strictly maintaining the **compact logarithmic proof size** ($O(\log N)$) characteristic of Bulletproofs. To achieve this, **Cougar** introduces a generalized two-tier commitment framework combined with a *disjoint interpolation* strategy for efficient consistency checks.

We implemented **Cougar** in Rust and performed a comprehensive benchmarking against Bulletproofs and **Leopard** (IEEE TIFS). Our evaluation demonstrates that while **Cougar** incurs a moderate increase in prover overhead, its verification time scales significantly better for large instances. Concretely, for a witness size of $N = 2^{20}$, **Cougar** achieves a $50\times$ verification speed-up over Bulletproofs and exhibits a superior asymptotic growth rate compared to existing sublinear IPAs.

INDEX TERMS Inner product argument, zero knowledge proof, polynomial commitment, discrete logarithm assumption,

I. INTRODUCTION

A ZKP is a cryptographic protocol that allows a prover to convince a verifier of a statement's truth without revealing any information beyond its validity [32]. ZKPs serve as a foundational component in numerous applications, including identification schemes [21], [27], verifiable computation [7], [9], [12], [56], and confidential transactions [16], [24], [31], [38], [52]. Among these, ZKPs for the satisfiability of Arithmetic Circuits (AC) are particularly crucial, and many constructions have been proposed to this end [6], [11], [16], [19], [22], [30], [33], [36].

One notable approach for constructing ZKPs under the Discrete Logarithm (DL) assumption involves using an Inner Product Argument (IPA) as a core building block [11], [16], [19], [24], [45]. Bootle et al. [11] first introduced an IPA

with logarithmic proof size under the DL assumption. This was later improved by Bünz et al. [16] in their work, Bulletproofs. Subsequently, Bünz et al. [19] established a paradigm for building ZKPs by applying a Polynomial Commitment Scheme (PCS), which is a specialized form of IPA, to a Polynomial Interactive Oracle Proof (PIOP) system. Within this paradigm, the overall efficiency of a ZKP system heavily depends on the underlying IPA, making the design of efficient IPAs a critical research area.

Bulletproofs is widely recognized for its efficient proof size and its reliance on a transparent setup, which does not require trusted parties. However, a major drawback of Bulletproofs is its linear-time verification cost, which poses a significant performance bottleneck in applications like verifiable and incrementally verifiable computation.

To address this challenge of linear verification, Daza et al. [25] proposed an IPA with a sublinear-time verifier using bilinear pairings. However, their construction requires a trusted setup to generate a Common Reference String (CRS), a requirement that Bulletproofs eliminates. This reliance on a trusted setup limits its applicability in decentralized settings like blockchains.

Alternatively, Lee [45] proposed a pairing-based IPA named Dory, which avoids a trusted setup. Nevertheless, Dory relies on stronger cryptographic assumptions, such as the Symmetric External Diffie-Hellman (SXDH) assumption, and requires significant precomputation.

Without introducing stronger assumptions than those used in Bulletproofs, Kim et al. [41] recently proposed two IPAs with square-root verifier complexity: a pairing-based IPA (Protocol3) and a pairing-free IPA (Protocol4). Both constructions feature a linear-time prover and logarithmic communication complexity, matching the efficiency of Bulletproofs. More recently, Kim et al. [40] presented optimizations and a concrete implementation of Protocol3, naming it Leopard.

A. OUR CONTRIBUTION

In this paper, we introduce the first cubic root verifier and logarithmic proof size IPA, called Cougar, under the DL assumption with a transparent setup. Our IPA maintains the same assumptions and setup as previous IPAs, such as [11], [16], [41]. In Table 1, we provide a comparison of computation and communication complexity between previous IPA proposals and ours.

To achieve a cubic root verifier, we thoroughly investigate the previous pairing-free IPA Protocol4 from the lens of a two-tier commitment scheme. From this, we present a generalization of Protocol4 and improve its verifier's complexity by combining with Leopard in the second layer. With these framework-level improvements, we also propose novel techniques to efficiently deal with relations between elliptic curve points in Protocol4. First, we present a Plonkish-based proof system tailored for these relations, which utilizes new custom gates for *mixed* elliptic curve group operations. Moreover, we present a Plonk-friendly encoding method for a homomorphic PCS, enabling an efficient consistency check between committed vectors and intermediate wire values in AC.

We prove that Cougar satisfies the claimed features under the DL assumption. Furthermore, we also conducted experimental analyses of Cougar by implementing it in our experimental environment. We compared Cougar with other previous IPAs under the same setting: BulletProofs and Leopard. We showed that the verification cost of Cougar increases much more slowly than these two prior works as the length of the witness vectors increases.

B. TECHNICAL OVERVIEW

1) Generalized Two-Tier Commitment

Our construction builds upon the two-tier commitment schemes introduced in [41]: a pairing-based homomorphic

commitment and a pairing-free, doubly-Pedersen commitment. A two-tier commitment consists of two layers. In the first layer, a vector of length mn is compressed into n intermediate commitments using an initial commitment scheme with an m -dimensional key. In the second layer, these n commitments are then recommitted using a second scheme with an $O(n)$ -dimensional key, compressing them into a single, constant-size commitment.

The efficiency of recent IPAs stems from this two-tier structure, which reduces the verifier cost to $O(m + n)$, a significant improvement compared to the linear cost, $O(mn)$ of Bulletproofs. To further reduce the verification cost by shrinking these dimensions, we introduce a *generalized two-tier commitment scheme*. Instead of relying on a specific commitment in the second layer as in [41], we generalize the framework to integrate a sublinear-verifier polynomial commitment scheme, effectively nesting the commitments.

2) Proof for Elliptic Curve Relation

Constructing an IPA with **additively homomorphic** commitments follows a paradigm similar to Bulletproofs. However, schemes that are not homomorphic in the standard sense require auxiliary proofs to validate reduction steps.

Although the doubly-Pedersen commitment is not additively homomorphic, it exhibits a *layer-wise homomorphic* property because each of its underlying layers is homomorphic. The authors of [41] leveraged this property to achieve a sublinear verifier cost in Protocol4, despite the overhead of auxiliary proofs.

To establish the correctness of elliptic curve relations in these proofs, we adopt the Plonkish proof system [64] rather than general-purpose RICS. Plonkish leverages **custom gates** tailored for group operations, offering significant efficiency gains. We instantiate this system by combining it with the Leopard_{PC} polynomial commitment scheme [40].

3) Cubic Root Verifier Inner Product Argument

By integrating the **generalized two-tier framework** with the Plonkish proof system, our protocol achieves a communication complexity of $O(\log_2 N)$ and a verifier complexity of $O(m + \|\mathcal{V}_{\text{Eval}}(n \log m)\|)$. Here, $\|\mathcal{V}_{\text{Eval}}(\cdot)\|$ denotes the verifier complexity of the evaluation protocol for the underlying Polynomial Commitment Scheme (PCS).

To instantiate this construction, we employ the evaluation protocol from Leopard [40], which features square-root verifier complexity. We then optimize the parameters by setting $m = \sqrt[3]{N}$ and $n = \sqrt[3]{N^2}$, where N is the witness length. This configuration yields a total verifier complexity of $O(\sqrt[3]{N} + \sqrt{n \log m}) = O(\sqrt[3]{N} + \sqrt{\sqrt[3]{N^2} \log(\sqrt[3]{N})}) = \tilde{O}(\sqrt[3]{N})$. This result represents an asymptotic cubic-root verifier complexity, marking a significant improvement over prior works.

C. RELATED WORKS

1) ZK Arguments from DL-based Inner Product Arguments
Bootle et al. [11], [16] established a framework to construct zero-knowledge arguments from IPA under the Dis-

TABLE 1. Comparison Table of IPAs for length- N vectors under Discrete Logarithm Setting

Schemes	Proof	Prover	Verifier	Assumption	Setup	Pairing
Updatable IPA [25]	$O(\log_2 N)$	$O(N)$	$O(\log_2 N)$	DL, DPair	Private	Yes
Dory [45]	$O(\log_2 N)$	$O(N)$	$O(\log_2 N)$	SXDH	Public	Yes
Bulletproofs [11], [16]	$O(\log_2 N)$	$O(N)$	$O(N)$	DL	Public	No
Leopard [40], [41]	$O(\log_2 N)$	$O(N)$	$O(\sqrt{N})$	DL	Public	Yes
Protocol4 [41]	$O(\log_2 N)$	$O(N)$	$O(\sqrt{N} \log_2 N)$	DL	Public	No
TENET [44]	$O(\sqrt{\log_2 N})$	$O(N \cdot 2^{\sqrt{\log_2 N}})$	$O(N/2^{\sqrt{\log_2 N}})$	DL, DPair	Public	Yes
This Work	$O(\log_2 N)$	$O(N)$	$O(\sqrt[3]{N} \sqrt{\log_2 N})$	DL	Public	Yes

Proof: proof size, **Prover:** prover complexity, **Verifier:** verifier complexity, **Assumption:** underlying cryptographic assumption, **Setup:** type of setup; private and public stand for trusted and transparent setup respectively, **Pair:** requirement of pairing-friendly groups

crete Logarithm (DL) assumption. This approach enables the design of efficient arguments characterized by a transparent setup and logarithmic proof size ($O(\log N)$). Subsequent research has focused on optimizing this proof size further; Bulletproofs+ [24] improved the concrete proof size of Bulletproofs, while other works such as Protocol2 [41] and TENET [44] achieved sub-logarithmic communication complexity ($O(\sqrt{\log N})$).

However, despite these improvements in communication, a significant drawback of these recursive arguments is their verification complexity. While standard schemes suffer from linear verification ($O(N)$), some recent works have addressed this bottleneck. For instance, Daza et al. [25] achieved logarithmic verification complexity but requires a trusted setup. TENET [44] achieved sublinear verification alongside sub-logarithmic proof size, and Dory [45] also reduced verification to logarithmic order. However, these efficient verification schemes typically necessitate trade-offs, such as relying on a trusted setup [25] or stronger assumptions like the Double Pairing (DPair) assumption [44] or the Symmetric External Diffie-Hellman (SXDH) assumption [45].

Under the standard DL assumption with a transparent setup, the most prominent progress in reducing verifier complexity has been the achievement of square-root order ($O(\sqrt{N})$), as demonstrated in schemes like Protocol4 [41] and Leopard [40]. Consequently, reducing the verifier complexity beyond this square-root barrier while maintaining the standard DL assumption remains a significant challenge.

2) ZK Arguments based on other Assumptions

To mitigate the high verifier cost inherent in the DL setting, alternative approaches based on groups of unknown order (UO) have been explored. Supersonic [19] achieves logarithmic verification complexity, while Dew [2] achieves constant-size communication. However, these constructions generally incur superlinear prover complexity or rely on RSA-based assumptions that typically require a trusted setup. While class groups can eliminate the trusted setup, they introduce significant computational overhead for the prover compared to standard DL groups.

Addressing the threat of quantum computing, several ZK arguments based on lattice assumptions (e.g., SIS, LWE) have been proposed [3], [4], [14], [18], [37], [46]. Although recent

works like HSS [37] and Greyhound [46] have improved efficiency, lattice-based schemes often suffer from larger communication costs compared to DL-based schemes due to the soundness slack or rejection sampling required for security. In contrast, our DL-based construction maintains compact logarithmic proof sizes, offering a practical advantage in bandwidth-constrained environments.

Another direction for post-quantum security involves using cryptographic hash functions [6], [8], [13], [61]. Since hash-based schemes do not rely on structured algebraic groups, they often achieve linear prover time [33], [61]. However, they typically result in polylogarithmic proof sizes ($O(\log^2 N)$), which are concretely larger than the logarithmic proofs ($O(\log N)$) of DL-based arguments. Furthermore, the lack of algebraic structure makes it challenging to apply advanced techniques such as proof aggregation or algebraic reduction, which are natural in our group-based setting.

3) Range Proof

Range proofs are a key application of IPAs. A range proof allows a prover to convince a verifier that a secret value lies within a specific range (e.g., $[0, 2^n - 1]$) without leaking any additional information. This primitive is a core cryptographic tool for enabling confidential transactions in blockchain systems [59], [62]. Following the paradigm of Bulletproofs [16], efficient range proofs can be constructed from an IPA.

Building on this foundation, several subsequent works have focused on further reducing the proof size of IPA-based range proofs. Bulletproofs+ [24] and Bulletproofs++ [26] introduced techniques to reduce the proof size and prover complexity. However, these optimizations typically inherit the linear verification cost of the underlying IPA, which can become a scalability bottleneck when verifying aggregated proofs for a large number of transactions.

II. PRELIMINARIES

A. NOTATIONS

We first define the notations used in this paper. We denote $[\ell]$ as the set of integers from 1 to ℓ , and $\text{negl}(\lambda)$ as a negligible function in the security parameter λ .

For a prime p , we consider asymmetric bilinear groups of order p , denoted by $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_t$, equipped with a non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_t$. For consistency, we employ additive notation to describe group operations

across all groups, including the target group $\mathbb{G}_t$. We generally use upper and lowercase letters to denote group elements and field elements, respectively. Bold font is used to represent vectors in $\mathbb{Z}_p^m$ or $\mathbb{G}^m$.

A summary of the mathematical notations and operations used throughout this paper is provided in Table 2.

TABLE 2. Summary of Notations

Symbol	Description
$[\ell]$	A set of integers $\{1, \dots, \ell\}$
$\text{negl}(\lambda)$	A negligible function
p	A prime order of groups
$\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t$	Asymmetric bilinear groups of order p
$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_t$	A non-degenerate bilinear map
$[k]G$	Scalar multiplication of point G by scalar $k \in \mathbb{Z}_p$
$\mathbf{a}, \mathbf{G}$	Vectors in $\mathbb{Z}_p^m$ and $\mathbb{G}^m$ (bold font)
a_i	The i -th element of a vector $\mathbf{a}$
$\mathbf{a} \parallel \mathbf{b}$	Concatenation of vectors $\mathbf{a}$ and $\mathbf{b}$
$\mathbf{a} + \mathbf{b}, \mathbf{G} + \mathbf{H}$	Component-wise addition
$\mathbf{a} \circ \mathbf{b}$	Component-wise product $(a_1 b_1, \dots, a_m b_m)$
$\langle \mathbf{a}, \mathbf{b} \rangle$	Inner product, $\sum a_i b_i$
$[x]G$	Multi-Scalar Multiplication (MSM), $\sum [x_i]G_i$
$E(\mathbf{G}, \mathbf{H})$	Inner Pairing Product, $\sum e(G_i, H_i)$
$[\mathbf{a}]G$	Parallel MSM for matrix $\mathbf{a} \in \mathbb{Z}_p^{m \times n}$ and $\mathbf{G} \in \mathbb{G}^m$

B. ARGUMENT OF KNOWLEDGE

Let $\mathcal{R}$ be a polynomial-time verifiable relation consisting of common reference string (CRS), statement, and witness, denoted by σ , x , and w respectively. An interactive argument system for relation $\mathcal{R}$ consists of three probabilistic polynomial-time algorithms (PPTs) $(\mathcal{K}, \mathcal{P}, \mathcal{V})$. The $\mathcal{K}$ algorithm takes the security parameter λ and outputs CRS σ , which is the input of $\mathcal{P}$ and $\mathcal{V}$. $\mathcal{P}$ and $\mathcal{V}$ generate a transcript interactively, denoted by $tr \leftarrow \langle \mathcal{P}(\sigma, x, w), \mathcal{V}(\sigma, x) \rangle$. At the end of the transcript, $\mathcal{V}$ outputs a bit, 0(reject) or 1(accept). An argument of knowledge (AoK) is a special case of an argument system that satisfies the properties of completeness and witness extractability.

Definition II.1 (Argument of Knowledge). *Let $\mathcal{R}$ be a polynomial-time verifiable relation. We call that the argument system $(\mathcal{K}, \mathcal{P}, \mathcal{V})$ for the relation $\mathcal{R}$ an Argument of knowledge if the following properties hold:*

[Completeness]: For every PPT adversary $\mathcal{A}$, the following inequality holds:

$$\Pr \left[\begin{array}{l} tr \leftarrow \langle \mathcal{P}(\sigma, x, w), \mathcal{V}(\sigma, x) \rangle \\ tr \text{ is accepting} \end{array} \middle| \begin{array}{l} \sigma \leftarrow \mathcal{K}(1^\lambda); \\ (x, w) \leftarrow \mathcal{A}(\sigma) \\ \wedge (\sigma, x; w) \in \mathcal{R} \end{array} \right] > 1 - \text{negl}(\lambda)$$

[Witness Extended Emulation]: For every deterministic polynomial prover $\mathcal{P}^$, which may not follow $\mathcal{P}$, and all pairs of PPT adversaries $(\mathcal{A}_1, \mathcal{A}_2)$, there exists a PPT*

emulator $\mathcal{E}$, the following inequality holds:

$$\Pr \left[\begin{array}{l} \mathcal{A}_1(tr) = 1 \\ \wedge \mathcal{A}_1(tr) = 1 \\ \wedge (\sigma, w, x) \in \mathcal{R} \end{array} \middle| \begin{array}{l} \sigma \leftarrow \mathcal{K}(1^\lambda); \\ (x, s) \leftarrow \mathcal{A}_2(\sigma); \\ tr \leftarrow \langle \mathcal{P}^*(\sigma, x, s), \mathcal{V}(\sigma, x) \rangle \\ \sigma \leftarrow \mathcal{K}(1^\lambda); \\ (x, s) \leftarrow \mathcal{A}_2(\sigma); \\ (tr, w) \leftarrow \mathcal{E}^\mathcal{O}(\sigma, x) \end{array} \right] < \text{negl}(\lambda)$$

The emulator $\mathcal{E}$ can access the oracle $\mathcal{O} = \langle \mathcal{P}^(\sigma, x, s), \mathcal{V}(\sigma, x) \rangle$, which outputs the transcript between $\mathcal{P}^*$ and $\mathcal{V}$. $\mathcal{E}$ permits to rewind $\mathcal{P}^*$ at a specific round and rerun $\mathcal{V}$ using fresh randomness. s can be considered as the state of $\mathcal{P}^*$.*

Trusted Setup. In some arguments, the key generation algorithm takes a trapdoor that should not be revealed to anyone, including the prover and verifier. In this case, CRS generation should be run by a trusted third party. A setting requiring a trusted party is called the trusted setup.

Definition II.2 (Discrete Logarithm Relation Assumption). *Let $\mathcal{G}$ be a group generator that takes security parameters λ and then outputs $\mathbb{G}$, describing a group of order p . We say that $\mathbb{G}$ satisfies the discrete logarithm relation (DLR) assumption if, for all non-uniform polynomial-time adversaries $\mathcal{A}$, the following inequality holds:*

$$\Pr \left[\begin{array}{l} \mathbf{a} \neq \mathbf{0} \\ \wedge [\mathbf{a}]G = 0_{\mathbb{G}} \end{array} \middle| \begin{array}{l} (p, g, \mathbb{G}) \leftarrow \mathcal{G}(1^\lambda), \mathbf{g} \xleftarrow{\$} \mathbb{G}^n; \\ \mathbf{a} \leftarrow \mathcal{A}(g, p, g, \mathbb{G}) \end{array} \right] \leq \text{negl}(\lambda)$$

It is well-known that the discrete logarithm relation (DLR) assumption is equivalent to the discrete logarithm (DL) assumption [16], [41].

Definition II.3 (Commitment Scheme). *A commitment scheme $\mathcal{C}$ consists of three PPT algorithms: a key generation Gen, a commitment Com, and an open Open. A commitment scheme $\mathcal{C} = (\text{Gen}, \text{Com}, \text{Open})$ over a message space $\mathbb{M}$, a random space $\mathbb{R}$, and a commitment space $\mathbb{C}$ is defined by:*

- $\text{Gen}(1^\lambda, \ell) \rightarrow \text{ck}$: On input security parameter λ and dimension of message space ℓ , sample commitment key ck
- $\text{Com}(\text{ck}, m; r) \rightarrow C$: Take commitment key ck , message $m \in \mathbb{M}$, and randomness $r \in \mathbb{R}$, output commitment $C \in \mathbb{C}$
- $\text{Open}(\text{ck}, m, r, C) \rightarrow 0/1$: Take commitment key ck , message $m \in \mathbb{M}$, randomness $r \in \mathbb{R}$, and commitment $C \in \mathbb{C}$ output 1 if $\text{Com}(\text{ck}, m; r) = C$, 0 otherwise.

Since the Open algorithm can be described by using Com algorithm, we omit the Open algorithm from the commitment scheme $\mathcal{C}$. Now, we call $\mathcal{C} = (\text{Gen}, \text{Com})$ a commitment scheme if the following properties hold:

[Binding]: For any expected PPT adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} C_0 = C_1 \\ \wedge m_0 \neq m_1 \end{array} \middle| \begin{array}{l} \text{ck} \leftarrow \text{Gen}(1^\lambda, \ell); \\ (m_0, r_0, m_1, r_1) \leftarrow \mathcal{A}(\text{ck}); \\ C_i \leftarrow \text{Com}(\text{ck}, m_i; r_i), i = 0, 1 \end{array} \right] \leq \text{negl}(\lambda)$$

[Hiding]: For any expected PPT adversary $\mathcal{A} =$

$$\Pr \left[\begin{array}{l} \mathbf{b} = \mathbf{b}' \end{array} \middle| \begin{array}{l} \text{ck} \leftarrow \text{Gen}(1^\lambda, \ell); \\ (m_0, m_1, \text{state}) \leftarrow \mathcal{A}_1(\text{ck}); \\ \mathbf{b} \xleftarrow{\$} \{0, 1\}, r \xleftarrow{\$} \mathbb{R}; \\ C \leftarrow \text{Com}(\text{ck}, m_b; r); \\ \mathbf{b}' \leftarrow \mathcal{A}_2(\text{ck}, C, \text{state}), \end{array} \right] - \frac{1}{2} \leq \text{negl}(\lambda)$$

Additionally, we call a commitment scheme $\mathcal{C}$ is (additively) homomorphic if the following property holds:

[(Additive) Homomorphic]: For any commitment key $ck \leftarrow \text{Gen}(1^\lambda, \ell)$ and pairs of message-randomness $(m_0, r_0), (m_1, r_1) \in \mathcal{M} \times \mathcal{R}$, the following equality holds:

$$\text{Com}(ck, m_0; r_0) + \text{Com}(ck, m_1; r_1) = \text{Com}(ck, m_0 + m_1; r_0 + r_1)$$

C. HOMOMORPHIC VECTOR COMMITMENT SCHEMES

A homomorphic vector commitment scheme is a homomorphic commitment for N -dimensional message, etc. $\mathbb{Z}_p^N$ or $\mathbb{G}^N$. We introduce two widely used homomorphic vector commitment schemes: *Pedersen vector commitment* [47] and *AFGHO group commitment* [1].

1) Pedersen vector commitment.

Pedersen vector commitment $\mathcal{C}_{\text{Ped}} = (\text{Gen}_{\text{Ped}}, \text{Com}_{\text{Ped}})$ is a commitment scheme over message space $\mathbb{Z}_p^N$ as follows:

- $\text{Gen}_{\text{Ped}}(1^\lambda, N) \rightarrow (\mathbf{G}, H)$:
 - 1) Sample $\mathbf{G} \xleftarrow{\$} \mathbb{G}^N$ and $H \xleftarrow{\$} \mathbb{G}$
 - 2) Output $ck = (\mathbf{G}, H) \in \mathbb{G}^N \times \mathbb{G}$
- $\text{Com}_{\text{Ped}}((\mathbf{G}, H), \mathbf{a}; r) \rightarrow C$:
 - 1) Compute $C = [\mathbf{a}]\mathbf{G} + [r]H$
 - 2) Output $C \in \mathbb{G}$

Specially, we sometimes use subscript Ped, p for Pedersen commitment over group $\mathbb{G}_p$ of order p to distinguish base group.

2) AFGHO group commitment.

AFGHO group commitment $\mathcal{C}_{\text{GC}} = (\text{Gen}_{\text{GC}}, \text{Com}_{\text{GC}})$ is a commitment scheme over message space $\mathbb{G}_1^N$ as follows:

- $\text{Gen}_{\text{GC}}(1^\lambda, N) \rightarrow (\mathbf{F}, K)$:
 - 1) Sample $\mathbf{F} \xleftarrow{\$} \mathbb{G}_2^N$ and $K \xleftarrow{\$} \mathbb{G}_t$
 - 2) Output $ck = (\mathbf{F}, K) \in \mathbb{G}_2^N \times \mathbb{G}_t$
- $\text{Com}_{\text{GC}}((\mathbf{F}, K), \mathbf{G}; r) \rightarrow C$:
 - 1) Compute $C = \mathbf{E}(\mathbf{G}, \mathbf{F}) + [r]K$
 - 2) Output $C \in \mathbb{G}_t$

D. POLYNOMIAL COMMITMENT SCHEME [19], [39]

A polynomial commitment scheme (PCS) is a special case of the commitment scheme that commits the given polynomial within the specific degree bound d . PCS allows convincing polynomial evaluation without opening the polynomial itself. Concretely, PCS contains an argument system $\text{Eval} = (\mathcal{K}, \mathcal{P}, \mathcal{V})$ for the following relation:

$$\mathcal{R}_{\text{Eval}} = \left\{ \left(\begin{array}{l} ck_{\text{PC}}, C \in \mathcal{C}, \\ z, y \in \mathbb{Z}_p, d \in \mathbb{N}; \\ f \in \mathbb{Z}_p^{< d}[X] \end{array} \right) : \begin{array}{l} C = \text{Com}(ck_{\text{PC}}, f) \\ \wedge y = f(z) \\ \wedge \deg(f) \leq d \end{array} \right\} \quad (1)$$

The formal definition of PCS is given as below:

Definition II.4 (Polynomial Commitment Scheme). A polynomial commitment scheme $\text{PCS} = (\text{Gen}, \text{Com}, \text{Eval})$ consists of key generation algorithms Gen , commitment algorithm Com , and argument system Eval for the relation $\mathcal{R}_{\text{Eval}}$.

We call $\text{PCS} = (\text{Gen}, \text{Com}, \text{Eval})$ is a polynomial commitment scheme if the following properties hold:

- The (Gen, Com) is commitment schemesatisfies the binding property.
- The argument system Eval is an AoK for the relation $\mathcal{R}_{\text{Eval}}$ in Eq. (1)

III. GENERALIZED TWO-TIER COMMITMENT SCHEME

A two-tier commitment scheme is a cryptographic primitive designed to commit to structured data, such as a matrix in $\mathbb{Z}_p^{m \times n}$, by hierarchically combining two underlying commitment schemes. This hierarchical structure is pivotal for constructing efficient IPAs, as it allows the Common Reference String (CRS) size to be reduced to sublinear complexity $O(n + m)$ relative to the total witness size $N = mn$, thereby significantly lowering verification costs [20], [40], [41], [45].

Structurally, a generalized two-tier commitment scheme consists of three main components: a first-tier commitment $\mathcal{C}_1$, an intermediate encoding function Ecd , and a second-tier commitment $\mathcal{C}_2$. The commitment process follows a pipeline: first, the matrix rows are committed using $\mathcal{C}_1$; second, the resulting commitments are transformed via Ecd into a format compatible with the second layer; finally, these encoded values are committed using $\mathcal{C}_2$.

Definition III.1 (Generalized Two-tier Commitment Scheme).

Let $\mathcal{C}_1 = (\text{Gen}_1, \text{Com}_1)$ be a commitment scheme defined over $(\mathbb{Z}_p^n, \mathcal{C}_1, \mathcal{R}_1)$. Let $\mathcal{C}_2 = (\text{Gen}_2, \text{Com}_2)$ be a commitment scheme defined over $(\mathcal{M}_2^\ell, \mathcal{C}_2, \mathcal{R}_2)$, where $\mathcal{M}_2$ is the base message space and ℓ is the dimension parameter.

Let $\text{Ecd} : \mathcal{C}_1^m \rightarrow \mathcal{M}_2^\ell$ be a deterministic encoding function. We require Ecd to be injective and efficiently invertible on its image. That is, for any $\mathbf{C} \in \mathcal{C}_1^m$, $\text{Ecd}(\mathbf{C})$ is unique, and there exists an efficient algorithm Dec such that $\text{Dec}(\text{Ecd}(\mathbf{C})) = \mathbf{C}$.

A commitment scheme $\mathcal{C} = (\text{Gen}, \text{Com})$ over the space $(\mathbb{Z}_p^{m \times n}, \mathcal{C}_2, \mathcal{R}_1^m \times \mathcal{R}_2)$ is called a **generalized two-tier commitment scheme** defined as follows:

- $\text{Gen}(1^\lambda, mn) \rightarrow ck = (ck_1, ck_2)$:
 - 1) Run $\text{Gen}_1(1^\lambda, n) \rightarrow ck_1$.
 - 2) Run $\text{Gen}_2(1^\lambda, \ell) \rightarrow ck_2$.
 - 3) Return $ck = (ck_1, ck_2)$.
- $\text{Com}(ck, M; (\mathbf{r}, r_f)) \rightarrow C$:
 - 1) Let $M_1, \dots, M_m \in \mathbb{Z}_p^n$ be the rows of matrix M .
 - 2) Parse $\mathbf{r} = (r_1, \dots, r_m) \in \mathcal{R}_1^m$.
 - 3) **Layer 1:** Compute $C_i \leftarrow \text{Com}_1(ck_1, M_i; r_i)$ for all $i \in [m]$. Let $\mathbf{C} = (C_1, \dots, C_m) \in \mathcal{C}_1^m$.
 - 4) **Encoding:** Compute the encoded vector $W \leftarrow \text{Ecd}(\mathbf{C}) \in \mathcal{M}_2^\ell$.
 - 5) **Layer 2:** Compute the commitment $C \leftarrow \text{Com}_2(ck_2, W; r_f)$.
 - 6) Return C .

Remark (Binding Preservation). The injectivity of Ecd is a critical requirement to preserve the computational binding property of the overall scheme. If Ecd were not injective, distinct intermediate commitments $\mathbf{C} \neq \mathbf{C}'$ could map to the

same encoded value W , allowing an adversary to equivocate on the first-layer values.

A. TWO-TIER COMMITMENT SCHEMES

a: Pairing-based Two-tier Commitment schemes

By setting C_1 as the Pedersen commitment over $\mathbb{G}_1$ and C_2 as the AFGHO commitment over $\mathbb{G}_2$, one obtains a fully homomorphic two-tier scheme [20]. Here, Ecd is the identity function, mapping $\mathbb{G}_1^m$ directly to $\mathbb{G}_1^m$ (implying $\mathcal{M}_2 = \mathbb{G}_1$ and $\ell = m$). Since the identity map is trivially bijective, the binding property is preserved.

b: Doubly-Pedersen Two-tier Commitment Scheme

To remove pairings, Kim et al. [41] utilized Pedersen commitments for both layers. Here, C_1 commits to vectors in $\mathbb{G}_p$ (elliptic curve points), and Ecd interprets these points as scalars in the base field $\mathbb{Z}_q$. Thus, $\mathcal{M}_2 = \mathbb{Z}_q$ and $\ell = 2m$. This mapping is injective (interpreting coordinate representation) and efficiently invertible.

B. HOMOMORPHIC COMMITMENT IN SECOND LAYER

To apply the folding reduction technique to an IPA based on the Doubly-Pedersen commitment, the prover is required to provide additional proofs [41]. These proofs are necessary to validate the group operations performed on the first-layer commitments, as the map between the two layers (interpreting points as scalars) is not a homomorphism. However, the construction of these proofs is significantly simplified by the *layer-wise homomorphic* property, particularly because the second-layer commitment remains homomorphic with respect to the encoded message space.

Crucially, a key function of the second layer is *compression*: it must transform a large vector of first-layer commitments into a compact element. To generalize the Doubly-Pedersen commitment, we propose replacing the second-layer Pedersen commitment with *any* homomorphic commitment scheme, utilizing an explicit encoding function to bridge the algebraic gap.

Formally, let $C_1 = (\text{Gen}_{\text{Ped}}, \text{Com}_{\text{Ped}})$ be a Pedersen commitment scheme over $(\mathbb{Z}_p^n, \mathbb{G}_p)$ for the first layer, where $\mathbb{G}_p = E(\mathbb{Z}_q)$. Let $C_2 = (\text{Gen}_2, \text{Com}_2)$ be a generic homomorphic commitment scheme over $(\mathcal{M}_2, \mathbb{C})$ for the second layer. We introduce a deterministic encoding function $\text{Ecd} : \mathbb{G}_p^m \rightarrow \mathcal{M}_2$ that maps the vector of first-layer commitments to the message space of the second layer (e.g., coordinate decomposition mapping to $\mathbb{Z}_q^{2m}$). We defer the detailed description of our two-tier commitment scheme $C_{\text{TC}} = (\text{Gen}_{\text{TC}}, \text{Com}_{\text{TC}})$ to Section IV-B.

IV. TWO-TIER COMMITMENT-BASED IPA

We focus on the construction of an IPA utilizing the two-tier commitment scheme C_{TC} for the relation defined below. Note that for matrices $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_p^{m \times n}$, we denote $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i,j} a_{i,j} b_{i,j}$.

$$\mathcal{R}_{\text{TC}}^{m,n} = \left\{ \left(\begin{array}{l} \mathbf{G}, \mathbf{H} \in \mathbb{G}_p^m, \text{ck}_2, \text{P} \in \mathbb{G}_q, c \in \mathbb{Z}_p; \\ \mathbf{a}, \mathbf{b} \in \mathbb{Z}_p^{m \times n} \\ \text{P} = \text{Com}_{\text{TC}}((\mathbf{G} \parallel \mathbf{H}, \text{ck}_2), \mathbf{a} \parallel \mathbf{b}) \\ \wedge c = \langle \mathbf{a}, \mathbf{b} \rangle \end{array} \right) : \right\} \quad (2)$$

The reduction phase recursively transforms the initial argument instance through two main steps¹:

- 1) **Row-reduction** (Protocol.Row): This step is applied recursively until the witness matrix is reduced to a single row ($m = 1$). Each step reduces an instance of $\mathcal{R}_{\text{TC}}^{m,n}$ to an instance of $\mathcal{R}_{\text{TC}}^{m/2,n}$.
- 2) **Column-reduction** (Protocol.Col): Once the row-reductions are complete, this step is applied recursively until the witness is reduced to a single column ($n = 1$). Each step reduces an instance of $\mathcal{R}_{\text{TC}}^{1,n}$ to an instance of $\mathcal{R}_{\text{TC}}^{1,n/2}$.

The MEC proof phase validates the aggregated elliptic curve relations generated throughout the reduction phase. In this final step, the prover and verifier execute the AggMEC protocol, utilizing Eval and Plonkish_{Eval} as subroutines. Notably, both subroutines incur a verifier complexity of $O(\|\mathcal{V}_{\text{Eval}}(n \log_2 m)\|)$. The entire process is illustrated in Figure 1.

A. REDUCTION PROTOCOL

In the reduction protocol, the prover and verifier recursively execute the reduction process, transforming an argument for size- N vectors into one for half-sized vectors. Unlike Bulletproofs [16] or Leopard [40], both parties must store the *transcript history* of the reduction steps. This is necessary because the verifier defers the validity check of the group operation relations between the received commitments until the final phase. Specifically, the states st_V and st_P record this history, which is subsequently used in the aggregated multi-elliptic curve operation proof, AggMEC. This final check guarantees that the inner values of the commitments formed during the reduction are valid.

We construct two types of reduction protocols: a row-wise reduction (Protocol.Row) and a column-wise reduction (Protocol.Col), both sharing equivalent core principles. In each reduction step, the prover $\mathcal{P}$ sends cross-term values c_L, c_R along with commitments L, R —which commit to the split witness vectors—to the verifier $\mathcal{V}$. Upon receiving a challenge x from $\mathcal{V}$, $\mathcal{P}$ computes and sends an updated instance commitment $\hat{P}$.

A key distinction from standard homomorphic commitment-based IPAs is that $\mathcal{V}$ cannot locally compute the updated instance $\hat{P}$ due to the lack of additive homomorphism in the two-tier scheme. Instead, $\mathcal{V}$ receives $\hat{P}$ from $\mathcal{P}$ and *postpones*

¹Formally, our reduction protocols do not individually satisfy the definition of a *knowledge reduction* from [42], as they omit the verification of elliptic curve relations at each step. Instead, these checks are deferred to a final, aggregated phase where the AggMEC protocol validates all relations from the entire reduction process simultaneously. Consequently, the overall protocol satisfies knowledge soundness.

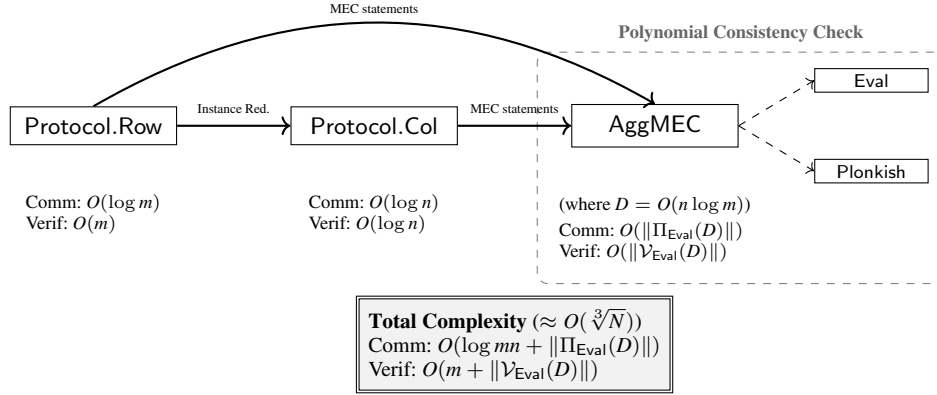


FIGURE 1. Overall Process of Cougar Protocol. The protocol recursively reduces the instance size (Row $\rightarrow$ Col). The MEC statements generated from both steps are aggregated and checked by AggMEC against polynomials of degree D (where D scales with $O(n \log m)$).

its verification. The correctness of this update is later verified in batch via AggMEC. We describe Protocol.Row and Protocol.Col in Algorithm 1 and Algorithm 2, respectively.

Algorithm 1 Protocol.Row

Protocol.Row($\mathbf{G}, \mathbf{H}, (\text{ck}_k)_{k=s}^\mu, \text{ck}_{\text{Col}}, \mathbf{P}, c, st_V; \mathbf{a}, \mathbf{b}, st_P$)
 where $\text{ck}_k = (\text{ck}_{L,k}, \text{ck}_{R,k}, \text{ck}_{P,k})$, $\text{ck}_{\text{Col}} = (\text{ck}_{P,k})_{k=\mu+1}^{\mu+\nu+1}$

- 1: **if** $m = 1$, base case $s = \mu$ **then**:
- 2: $\mathcal{P}$ and $\mathcal{V}$ run Protocol.Col($G, H, \text{ck}_{\text{Col}}, \mathbf{P}, c, st_V; \mathbf{a}, \mathbf{b}, st_P$)
- 3: **else**
- 4: **if** $st_P = \perp$ and $st_V = \perp$ **then**
- 5: $\mathcal{P}$ sets $\mathbf{P} = [\mathbf{a}|\mathbf{G}] \parallel [\mathbf{b}|\mathbf{H}]$ and adds $(\cdot, \cdot, \mathbf{P})$ into the bottom row of st_P .
- 6: $\mathcal{V}$ adds $(\text{ck}_{P,0}, \cdot, \cdot, \mathbf{P}, \cdot)$ into the bottom row of st_V .
- 7: **else**
- 8: $\mathcal{P}$ refers $\mathbf{P}$ in the bottom row of st_P
- 9: **end if**
- 10: Set $\hat{m} = \frac{m}{2}$ and $\mathbf{a} = [\mathbf{a}_L|\mathbf{a}_R]$, $\mathbf{b} = [\mathbf{b}_L|\mathbf{b}_R]$,
 $\mathbf{G} = \mathbf{G}_L \parallel \mathbf{G}_R$, $\mathbf{H} = \mathbf{H}_L \parallel \mathbf{H}_R$
- 11: $\mathcal{P}$ computes c_L, c_R and L, R and sends them to $\mathcal{V}$:
 $\mathbf{L} = [\mathbf{a}_L]\mathbf{G}_R \parallel [\mathbf{b}_R]\mathbf{H}_L, \mathbf{R} = [\mathbf{a}_R]\mathbf{G}_L \parallel [\mathbf{b}_L]\mathbf{H}_R \in \mathbb{G}_p^{2n}$,
 $c_L = \langle \mathbf{a}_L, \mathbf{b}_R \rangle, c_R = \langle \mathbf{a}_R, \mathbf{b}_L \rangle \in \mathbb{Z}_p$,
 $L = \text{Com}_2(\text{ck}_{L,s}, \text{Ecd}_{L,s}^{\text{row}}(\mathbf{L})) \in \mathbb{G}_q$,
 $R = \text{Com}_2(\text{ck}_{R,s}, \text{Ecd}_{R,s}^{\text{row}}(\mathbf{R})) \in \mathbb{G}_q$
- 12: $\mathcal{V}$ chooses $x \xleftarrow{\$} \mathbb{Z}_p^*$ and returns it to $\mathcal{P}$
- 13: $\mathcal{P}$ computes $\hat{\mathbf{P}}$ and sends it to $\mathcal{V}$:
 $\hat{\mathbf{P}} = [x^{-1}]\mathbf{L} + \mathbf{P} + [x]\mathbf{R} \in \mathbb{G}_p^{2n}$,
 $\hat{\mathbf{P}} = \text{Com}_2(\text{ck}_{P,s}, \text{Ecd}_{P,s}^{\text{row}}(\hat{\mathbf{P}})) \in \mathbb{G}_q$
- 14: Both $\mathcal{P}$ and $\mathcal{V}$ update:
 $\hat{\mathbf{G}} = \mathbf{G}_L + [x^{-1}]\mathbf{G}_R, \hat{\mathbf{H}} = \mathbf{H}_L + [x]\mathbf{H}_R \in \mathbb{G}_p^{\hat{m}}$,
 $\hat{c} = x^{-1}c_L + c + xc_R \in \mathbb{Z}_p$
- 15: $\mathcal{P}$ updates $\hat{\mathbf{a}} = \mathbf{a}_L + x\mathbf{a}_R, \hat{\mathbf{b}} = \mathbf{b}_L + x^{-1}\mathbf{b}_R \in \mathbb{Z}_p^{\hat{m} \times n}$.
- 16: $\mathcal{V}$ adds $(\text{ck}_s, L, R, \hat{\mathbf{P}}, x)$ into the bottom row of st_V .
- 17: $\mathcal{P}$ adds $(\mathbf{L}, \mathbf{R}, \hat{\mathbf{P}})$ into the bottom row of st_P .
- 18: Both $\mathcal{P}$ and $\mathcal{V}$ run the following protocol:
- 19: Protocol.Row($\hat{\mathbf{G}}, \hat{\mathbf{H}}, (\text{ck}_k)_{k=s+1}^\mu, \text{ck}_{\text{Col}}, \hat{\mathbf{P}}, \hat{c}, st_V; \hat{\mathbf{a}}, \hat{\mathbf{b}}, st_P$)
- 20: **end if**

Theorem IV.1. Assume that Protocol.Col provides both perfect completeness and computational witness-extended emulation. Then, Protocol.Row in Algorithm 1 satisfies perfect completeness and computational witness-extended emulation under the DL assumption.

Proof Sketch. Completeness follows from the homomorphic properties of the underlying algebraic structure. For witness-extended emulation, we construct a knowledge extractor that utilizes the General Forking Lemma [11]. By rewinding the adversary and obtaining transcripts for distinct challenges, the extractor can solve the linear system to recover the witness vectors. We provide the rigorous proof in Section A-B. $\square$

Theorem IV.2. Assume that AggMEC provides perfect completeness and computational witness-extended emulation. Then, Protocol.Col in Algorithm 2 satisfies perfect completeness and computational witness-extended emulation under the DL assumption.

Proof Sketch. Similar to the row-wise case, soundness relies on the DL assumption. We demonstrate that an extractor can recover the witnesses by leveraging the linear independence of the challenges in the reduction steps. The full proof is detailed in Section A-C. $\square$

B. AGGREGATED MULTI-ELLIPTIC CURVE OPERATION PROOF (AGGMEC)

From the results of Theorem IV.1 and Theorem IV.2, constructing a secure IPA for the relation $\mathcal{R}_{\text{TC}}^{m,n}$ (Eq. 2) reduces to designing an AggMEC protocol. The primary goal of AggMEC is to validate the correctness of the updated instances $\hat{\mathbf{P}}$ generated throughout the recursive reduction steps.

Recall that during the reduction, the verifier and prover maintain state tuples $(st_V; st_P)$ recording the history of transactions. Specifically, for each step k , the stored tuple corresponds to $(\text{ck}_k, (L_k, R_k, P_k, x_k); (L_k, R_k, P_k))$. AggMEC aggregates these records and verifies that they satisfy the following two categories of relations:

Algorithm 2 Protocol.Col

Protocol.Col($G, H, (ck_{P,k+\mu})_{k=s}^\nu, P, c, st_V; a, b, st_P$)

- 1: **if** $n = 1$, base case $s = \nu$ **then**:
- 2: $\mathcal{P}$ sends a and b to $\mathcal{V}$
- 3: $\mathcal{V}$ checks $c \stackrel{?}{=} a \cdot b$ and set $\mathbf{P}_{\text{Pub}} = [a]G \parallel [b]H \in \mathbb{Z}_q^4$
- 4: $\mathcal{P}$ and $\mathcal{V}$ run AggMEC($\mathbf{P}_{\text{Pub}}, st_V; st_P$)
- 5: **else**
- 6: **if** $st_P = \perp$ and $st_V = \perp$ **then**
- 7: $\mathcal{P}$ sets $\mathbf{P} = [a]G \parallel [b]H$ and adds $(\mathbf{P})$ into the bottom row of st_P
- $\mathcal{V}$ adds $(ck_{P,\mu}, P, \cdot)$ into the bottom row of st_V .
- 8: **else**
- 9: $\mathcal{P}$ refers $\mathbf{P}$ in the bottom row of st_P
- 10: **end if**
- Set $\hat{n} = \frac{n}{2}$ and $\mathbf{a} = \mathbf{a}_L \parallel \mathbf{a}_R$, $\mathbf{b} = \mathbf{b}_L \parallel \mathbf{b}_R$, $\mathbf{P} = \mathbf{P}^{(q_1)} \parallel \mathbf{P}^{(q_2)} \parallel \mathbf{P}^{(q_3)} \parallel \mathbf{P}^{(q_4)}$
- 11: $\mathcal{P}$ computes c_L and c_R and sends them to $\mathcal{V}$:
 $c_L = \langle \mathbf{a}_L, \mathbf{b}_R \rangle \in \mathbb{Z}_p$, $c_R = \langle \mathbf{a}_R, \mathbf{b}_L \rangle \in \mathbb{Z}_p$.
- 12: $\mathcal{V}$ chooses $x \xleftarrow{\$} \mathbb{Z}_p^*$ and returns it to $\mathcal{P}$
- 13: $\mathcal{P}$ computes $\hat{\mathbf{P}}$ and sends it to $\mathcal{V}$:
 $\hat{\mathbf{P}} = (\mathbf{P}^{(q_1)} + [x]\mathbf{P}^{(q_2)} \parallel \mathbf{P}^{(q_3)} + [x^{-1}]\mathbf{P}^{(q_4)}) \in \mathbb{G}_p^{2\hat{n}}$,
 $\hat{\mathbf{P}} = \text{Com}_2(ck_{P,\mu+s}, \text{Ecd}_{P,\mu+s}^{\text{col}}(\hat{\mathbf{P}})) \in \mathbb{G}_q$
- 14: Both $\mathcal{P}$ and $\mathcal{V}$ compute $\hat{c} = x^{-1}c_L + c + xc_R \in \mathbb{Z}_p$
- 15: $\mathcal{P}$ updates $\hat{\mathbf{a}} = \mathbf{a}_L + x\mathbf{a}_R$, $\hat{\mathbf{b}} = \mathbf{b}_L + x^{-1}\mathbf{b}_R \in \mathbb{Z}_p^{\hat{n}}$.
- 16: $\mathcal{V}$ adds $(ck_{P,\mu+s}, \hat{\mathbf{P}}, x)$ into the bottom row of st_V .
- 17: $\mathcal{P}$ adds $(\hat{\mathbf{P}})$ into the bottom row of st_P .
- 18: Both $\mathcal{P}$ and $\mathcal{V}$ run the following protocol:
- 19: Protocol.Col($G, H, (ck_{P,k+\mu})_{k=s+1}^\nu, \hat{\mathbf{P}}, \hat{c}, st_V; \hat{\mathbf{a}}, \hat{\mathbf{b}}, st_P$)
- 20: **end if**

1) **Consistency of Two-Tier Commitments:** The underlying vectors $\mathbf{L}_k, \mathbf{R}_k, \mathbf{P}_k$ must be valid openings of the second-tier commitments L_k, R_k, P_k stored by the verifier.

$$\begin{aligned} L_k &= \text{Com}_2(ck_{L,k}, \mathbf{L}_k), & \text{for } k = 1, \dots, \mu \\ R_k &= \text{Com}_2(ck_{R,k}, \mathbf{R}_k), & \text{for } k = 1, \dots, \mu \\ P_k &= \text{Com}_2(ck_{P,k}, \mathbf{P}_k), & \text{for } k = 0, \dots, \mu + \nu - 1 \\ P_{\mu+\nu} &= \text{Com}_2(ck, [a]G \parallel [b]H) & (\text{Base case}) \end{aligned} \quad (3)$$

2) **Elliptic Curve Operations on $\mathbb{G}_p = E(\mathbb{Z}_q)$:** The vectors must satisfy the homomorphic split-and-fold relations dictated by the reduction protocols.

$$\bigwedge_{k=0}^{\mu-1} (\mathbf{P}_{k+1} = [x_k^{-1}]\mathbf{L}_{k+1} + \mathbf{P}_k + [x_k]\mathbf{R}_{k+1}) \quad (4)$$

$$\bigwedge_{k=\mu}^{\mu+\nu-1} (\mathbf{P}_{k+1} = (\mathbf{P}_k^{(q_1)} + [x_k]\mathbf{P}_k^{(q_2)}) \parallel (\mathbf{P}_k^{(q_3)} + [x_k^{-1}]\mathbf{P}_k^{(q_4)})) \quad (5)$$

1) Plonkish: Proof System for Elliptic Curve Relations

Plonk [30] is a widely recognized method for representing the satisfiability of an arithmetic circuit (AC) as a constraint system. By leveraging Lagrange interpolation, circuit constraints are converted into polynomial identities, which can be efficiently proven using a Polynomial Interactive Oracle

Proof (PIOP) instantiated with a Polynomial Commitment Scheme (PCS) [19].

Plonkish [64] extends the standard Plonk arithmetization by supporting **custom gates** beyond simple addition and multiplication. This flexibility allows for the construction of highly optimized constraint systems over execution traces, particularly for complex algebraic operations such as elliptic curve group laws. We defer the detailed design of our execution traces and custom gates to Section C-B.

Throughout this paper, we denote $\text{Plonkish}_{\text{Eval}}$ as the specific proof system for verifying elliptic curve operations, utilizing the Plonkish arithmetization combined with the evaluation protocol of the underlying PCS (Eval). It accepts the following as public inputs: a commitment key ck_{PC} for the underlying PCS, selector polynomials $\{s_i(X)\}_{i=0}^{N_g-1}$, gate polynomials $\{g_i(X_1, \dots, X_M)\}_{i=0}^{N_g-1}$, and permutation polynomials $\{r_i(X)\}_{i=0}^{M-1}$. For the witness, $\text{Plonkish}_{\text{Eval}}$ takes wire polynomials $\{w^{(i)}(X)\}_{i=0}^{M-1}$, which encode the intermediate values (execution trace) of the elliptic curve operations. The detailed procedure of $\text{Plonkish}_{\text{Eval}}$ is provided in Algorithm 5 in the Section C-D.

2) Two Roots of Unity

To facilitate the consistency check between the two-tier commitments and the Plonkish execution trace, we employ two distinct domains defined by roots of unity. This approach allows us to encode the message vectors into polynomials that are compatible with the Plonk wiring check.

First, we determine the required size of the domains. Let d be the total number of group elements in the commitments $\mathbf{L}_k, \mathbf{R}_k$, and $\mathbf{P}_k$ across all reduction steps. Based on the row reduction, each $\mathbf{L}_k, \mathbf{R}_k$ consists of $2n$ group elements for $k = 1, \dots, \mu$, and $\mathbf{P}_k \in \mathbb{G}^{2n}$ consists of $2n$ group elements for $k = 0, \dots, \mu$. From the column reduction, each $\mathbf{p}_k$ consists of $n/2^{k-\mu-1}$ group elements for $k = \mu + 1, \dots, \mu + \nu$. Consequently, the total number of group elements is given by $d = 6\mu n + 4n - 2$.

We construct two polynomials, denoted as $F^{(x)}(X)$ and $F^{(y)}(X)$, that interpolate the x and y coordinates of these d group components, respectively, over the domain generated by the d -th root of unity ζ .

Next, we consider the domain size D for the execution trace. In our Plonkish setting, separate witness polynomials, denoted as $w^{(x)}(X)$ and $w^{(y)}(X)$, are generated for the x -coordinate and y -coordinate traces, respectively. Since the elliptic curve operations described in Eq. (4) and (5) require approximately $8n(\mu+1) \log_2 q$ rows in each trace, we choose an integer D such that $D \geq 8n(\mu+1) \log_2 q$ to interpolate these traces.

To verify that the execution trace consistently includes the coordinates of the group elements from the commitments, we check whether the values of the witness polynomials match the encoding polynomials over the commitment domain. Specifically, for each coordinate $c \in \{x, y\}$, the condition $w^{(c)}(\zeta^i) = F^{(c)}(\zeta^i)$ for all $i = 1, \dots, d$ is checked by

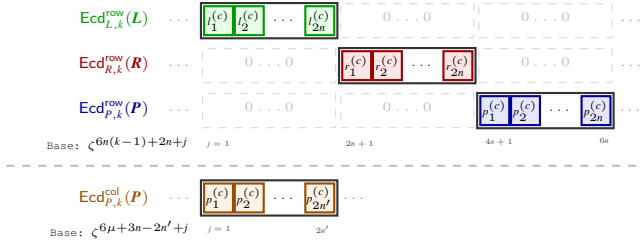


FIGURE 2. Visual representation of the encoding process.

verifying that the vanishing polynomial $Z_H(X) = X^d - 1$ divides the difference polynomial $w^{(c)}(X) - F^{(c)}(X)$.

To enable this polynomial divisibility check, we choose D such that $d \mid D$ (i.e., $D = t \cdot d$ for some integer t) and let ξ be the primitive D -th root of unity. By setting $\zeta = \xi^t$, the subgroup generated by ζ becomes a subgroup of the one generated by ξ , effectively embedding the commitment domain into the trace domain.

a: Encoding Process

To facilitate the consistency check between the vector of group elements $\mathbf{v} \in \mathbb{G}_p^{2n}$ and the wire polynomials interpolated over ξ^i , we employ a *disjoint interpolation* strategy for $\mathbf{v}$ over the domain generated by ζ^i . Since the wire polynomials are defined over the scalar field $\mathbb{Z}_q$, the encoding function must explicitly handle the conversion from group elements to their scalar representations.

We define the row encoding function $\text{Ecd}_{\text{type}}^{\text{row}} : \mathbb{G}_p^{2n} \rightarrow \mathbb{Z}_q[X] \times \mathbb{Z}_q[X]$, which takes a vector of group elements as input and outputs a pair of polynomials $(F^{(1)}(X), F^{(2)}(X))$, corresponding to the affine x and y coordinates, respectively.

- **Row Encoding Process:** Given an input vector $\mathbf{v} \in \mathbb{G}_p^{2n}$, the function first parses each element v_j into its affine coordinates $(v_j^{(1)}, v_j^{(2)}) \in \mathbb{Z}_q^2$ for $1 \leq j \leq 2n$. The non-zero evaluations of the polynomials $F^{(c)}(X)$ for each coordinate $c \in \{1, 2\}$ are determined by the type ($\text{type} \in \{L, R, P\}$) and the reduction step k :

- 1) **Type P** ($\text{Ecd}_{P,k}^{\text{row}}$): If $k > \mu$, it outputs $(0, 0)$. Otherwise, the coordinate $v_j^{(c)}$ is mapped to index $i = 6nk + j$.

$$F_{P,k}^{(c)}(\zeta^{3nk+j}) = v_j^{(c)} \quad \text{for } c \in \{1, 2\}$$

- 2) **Type L** ($\text{Ecd}_{L,k}^{\text{row}}$): If $k = 0$ or $k > \mu$, it outputs $(0, 0)$. Otherwise, the coordinate $v_j^{(c)}$ is mapped to index $i = 6n(k-1) + 2n + j$.

$$F_{L,k}^{(c)}(\zeta^{6n(k-1)+2n+j}) = v_j^{(c)} \quad \text{for } c \in \{1, 2\}$$

- 3) **Type R** ($\text{Ecd}_{R,k}^{\text{row}}$): If $k = 0$ or $k > \mu$, it outputs $(0, 0)$. Otherwise, the coordinate $v_j^{(c)}$ is mapped to index $i = 6n(k-1) + 4n + j$.

$$F_{R,k}^{(c)}(\zeta^{6n(k-1)+4n+j}) = v_j^{(c)} \quad \text{for } c \in \{1, 2\}$$

In contrast to row encoding, the column encoding processes a vector of $2n'$ group elements, where $n' = n/2^{k-\mu-1}$.

This process exclusively involves type P , as only the vector P_k is defined for the reduction steps $k = \mu+1, \dots, \mu+\nu$. The column encoding function $\text{Ecd}_{P,k}^{\text{col}} : \mathbb{G}_p^{2n'} \rightarrow \mathbb{Z}_q[X] \times \mathbb{Z}_q[X]$ operates as follows:

- **Column Encoding Process:** Given an input vector $\mathbf{v} \in \mathbb{G}_p^{2n'}$, we decompose each element into its affine coordinates $(v_j^{(1)}, v_j^{(2)})$. For each $1 \leq j \leq 2n'$ and coordinate $c \in \{1, 2\}$:

- 1) **Type P** ($\text{Ecd}_{P,k}^{\text{col}}$): If $k \leq \mu$ or $k > \mu + \nu$, it outputs $(0, 0)$. Otherwise, the coordinate $v_j^{(c)}$ is placed at index $i = 6\mu + 3n - 2n' + j$.

$$F_{P,k}^{(c)}(\zeta^{6\mu+3n-2n'+j}) = v_j^{(c)}$$

For all other indices i not specified above, $F^{(c)}(\zeta^i) = 0$. This strategy ensures disjoint supports over the domain $\langle \zeta \rangle$, i.e.,

$$F_{\text{type},k}^{(c)}(\zeta^i) \cdot F_{\text{type}',k'}^{(c)}(\zeta^i) = 0 \quad \text{if } (\text{type}, k) \neq (\text{type}', k').$$

This property guarantees that the aggregated polynomial uniquely represents the circuit wires without interference. We illustrate this process in Figure 2.

b: Homomorphic Polynomial Commitment Scheme

Based on the encoding process defined above, we construct a two-tier commitment scheme. We employ a homomorphic polynomial commitment scheme Com_P for the second tier Com_2 . This function takes the encoded polynomials $(F^{(1)}, F^{(2)})$ as input and outputs their respective commitments $F^{(1)}$ and $F^{(2)}$.

The two-tier commitment operates in the following sequence: First, it commits to the input message matrix column-wise, producing a vector of group elements in $\mathbb{G}_p$. Next, it parses these intermediate commitments into coordinate vectors and maps each to a specific disjoint domain on the polynomials based on the parameters (type, k) . Finally, it commits to the resulting polynomials.

We define the commitment key for a specific vector type and reduction step as $\text{ck}_{\text{type},k} := (\text{type}, k, \text{ck}_{PC})$, where ck_{PC} is the commitment key for the underlying PCS, Com_{PC} . The complete commitment procedure is defined as follows:

- $\text{Com}_{TC}((\mathbf{G}, \text{ck}_{\text{type},k}), \mathbf{A} \in \mathbb{Z}^{m \times n}) \rightarrow \mathbf{C}$:
 - 1) **Layer 1 (Column-wise Commitment):** For each column vector $\mathbf{a}_i$ ($i \in [n]$) of the matrix $\mathbf{A}$, compute $C_i \leftarrow \text{Com}_{\text{Ped},p}(\mathbf{G}, \mathbf{a}_i) \in \mathbb{G}_p$. Let $\mathbf{C} = (C_1, \dots, C_n)$ be the vector of these intermediate commitments.
 - 2) **Encoding:** Parse $\mathbf{C}$ into coordinate polynomials. If $k \leq \mu$, compute $(F^{(1)}, F^{(2)}) \leftarrow \text{Ecd}_{\text{type},k}^{\text{row}}(\mathbf{C})$. Otherwise, compute $(F^{(1)}, F^{(2)}) \leftarrow \text{Ecd}_{P,k}^{\text{col}}(\mathbf{C})$.
 - 3) **Layer 2 (Polynomial Commitment):** Compute the commitment $(F^{(1)}, F^{(2)}) \leftarrow \text{Com}_2(\text{ck}_{\text{type},k}, (F^{(1)}, F^{(2)}))$, where $F^{(i)} \leftarrow \text{Com}_{PC}(\text{ck}_{PC}, F^{(i)})$.
 - 4) Return $(F^{(1)}, F^{(2)})$.

3) Consistency Proof.

Recall that the goal of AggMEC is to substantiate the relations in Eq. (3) and Eqs. (4), (5). Since the satisfaction of the latter

two relations is guaranteed by the underlying Plonkish_{Eval} protocol, our focus is restricted to verifying Eq. (3) and ensuring the consistency between the vector commitments $\{L_k, R_k, P_k\}$ and the wire polynomial of the execution trace.

To verify the consistency of the vectors L_k, R_k, P_k , we first aggregate the individual commitments L_k, R_k, P_k into a single commitment A . The underlying message polynomial of A is defined as the sum of the encoding polynomials: $a(X) = \sum F_{\text{type},k}(X)$. Consequently, consistency implies that the difference polynomial $w(X) - a(X)$ is divisible by the vanishing polynomial $Z_H(X) = X^d - 1$, since $w(\xi^i) = a(\xi^i)$ must hold for all indices i in the domain. The verifier checks this divisibility relation via an evaluation query to PCS, given the commitment to the wire polynomial $w(X)$.

Furthermore, we employ V and F_V to formally establish Eq. (3), ensuring that each commitment L_k, R_k, P_k correctly corresponds to its respective vector L_k, R_k, P_k . Using a random challenge ρ sampled by the verifier, these commitments and their corresponding encoding polynomials are aggregated into V and F_V , respectively.

Finally, we batch these consistency checks into a single polynomial equation by constructing P and F_P using a second random challenge τ . The verifier can then check all conditions simultaneously by opening the polynomials at a random point z . By the Schwartz-Zippel Lemma, satisfying this check guarantees the validity of the relations with negligible soundness error. Based on these techniques, we present the complete AggMEC protocol in Algorithm 3.

Theorem IV.3. Assume that the polynomial commitment scheme $\text{PCS} = (\text{Gen}, \text{Comp}_{\text{PC}}, \text{Eval})$ satisfies the homomorphic property and all properties specified in Definition II.4. Then, the AggMEC protocol described in Algorithm 3 satisfies perfect completeness and computational witness-extended emulation.

Proof Sketch. Completeness. The perfect completeness follows directly from the homomorphic properties of the underlying polynomial commitment scheme and the correctness of the sub-protocols Eval and Plonkish_{Eval}. If an honest prover follows the protocol with valid witnesses, all verification equations hold with probability 1.

Witness-Extended Emulation. To establish computational witness-extended emulation, we construct a polynomial-time extractor $\mathcal{E}$ that recovers the witness by interacting with the prover. The extraction strategy relies on the general forking lemma [16] and proceeds in a hierarchical manner.

First, $\mathcal{E}$ invokes the extractors for the underlying PCS.Eval and Plonkish_{Eval} protocols as subroutines. Since Plonkish_{Eval} is an Argument of Knowledge, this step yields the valid wire polynomials $\{w^{(i)}(X)\}_{i=1}^M$.

Next, $\mathcal{E}$ rewinds the prover to obtain a tree of accepting transcripts with distinct challenges.

• **Aggregated Consistency Check:** By collecting a sufficient number of transcripts with distinct values for the aggregation challenge τ , $\mathcal{E}$ interpolates the transcript values

Algorithm 3 AggMEC

AggMEC($P_{\text{Pub}}, \text{ck}_k, (L_k, R_k, P_k, x_k); (L_k, R_k, P_k)$)
 $\text{ck}_k = (\text{ck}_{L,k}, \text{ck}_{R,k}, \text{ck}_{P,k})$, each ck_k contains ck_{PC}

1: For $i = 1, 2, P$ and V set

$$A^{(i)} = \sum_{k=1}^{\mu} (L_k^{(i)} + R_k^{(i)}) + \sum_{k=0}^{\mu+\nu} P_k^{(i)}$$

2: For $i = 1, 2, P$ sets

$$a^{(i)} = \sum_{k=1}^{\mu} (F_{L,k}^{(i)} + F_{R,k}^{(i)}) + \sum_{k=0}^{\mu+\nu} F_{P,k}^{(i)}$$

$$F_{L,k}^{(i)} = \text{Ecd}_{L,k}^{\text{row}}(L_k^{(i)}) \text{ for } k = 1, \dots, \mu$$

$$F_{R,k}^{(i)} = \text{Ecd}_{R,k}^{\text{row}}(R_k^{(i)}) \text{ for } k = 1, \dots, \mu$$

$$F_{P,k}^{(i)} = \text{Ecd}_{P,k}^{\text{row}}(P_k^{(i)}) \text{ for } k = 0, \dots, \mu$$

$$F_{P,k}^{(i)} = \text{Ecd}_{P,k}^{\text{col}}(P_k^{(i)}) \text{ for } k = \mu + 1, \dots, \mu + \nu$$

3: $\mathcal{P}$ construct wire polynomials $\{w^{(i)}(X)\}_{i=1}^2$ from execution table with public in/out P_{Pub} and then computes $W^{(1)}, W^{(2)}, Q^{(1)}, Q^{(2)}$ and sends them to $\mathcal{V}$:

$$q^{(i)}(X) = \frac{w^{(i)}(X) - a^{(i)}(X)}{X^d - 1},$$

$$W^{(i)} = \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, w^{(i)}), Q^{(i)} = \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, q^{(i)})$$

4: $\mathcal{V}$ chooses $z, \rho \xleftarrow{\$} \mathbb{Z}_q$ and sends them to $\mathcal{P}$.

5: $\mathcal{P}$ and $\mathcal{V}$ compute:

$$V = \sum_{i=1}^2 \left(\sum_{k=1}^{\mu} ([\rho^{4k-2-i}] L_k^{(i)} + [\rho^{4k-i}] R_k^{(i)}) + \rho^{4\mu} \left(\sum_{k=0}^{\mu+\nu} [\rho^{2k-1+i}] P_k^{(i)} \right) \right)$$

6: $\mathcal{P}$ computes $F_V(X)$:

$$F_V = \sum_{i=1}^2 \left(\sum_{k=1}^{\mu} (\rho^{4k-2-i} F_{L,k}^{(i)} + \rho^{4k-i} F_{R,k}^{(i)}) + \rho^{4\mu} \left(\sum_{k=0}^{\mu+\nu} \rho^{2k-1+i} F_{P,k}^{(i)} \right) \right)$$

7: $\mathcal{P}$ sends $s, t^{(1)}, t^{(2)}, r^{(1)}, r^{(2)}$ to $\mathcal{V}$:

$$s = F_V(z), t^{(i)} = q^{(i)}(z), r^{(i)} = w^{(i)}(z)$$

8: $\mathcal{V}$ chooses $\tau \xleftarrow{\$} \mathbb{Z}_q$ and sends them to $\mathcal{P}$.

9: $\mathcal{P}$ and $\mathcal{V}$ set $P = V + \sum_{i=1}^2 ([\tau^i] A^{(i)} + [\tau^{2+i}] Q^{(i)})$ and $y = s + \sum_{i=1}^2 (\tau^i (r^{(i)} - t^{(i)}(z^d - 1)) + \tau^{2+i} t^{(i)})$

10: $\mathcal{P}$ set $F_P = F_V + \sum_{i=1}^2 (\tau^i a^{(i)} + \tau^{2+i} q^{(i)})$

11: $\mathcal{P}$ sets wire polynomials $\{w^{(i)}\}_{i=1}^M \in \mathbb{Z}_q[X]$ containing $w^{(1)}$ and $w^{(2)}$.

12: $\mathcal{P}$ and $\mathcal{V}$ run Eval($\text{ck}_{\text{PC}}, P, z, y; F_P$) and Eval($\text{ck}_{\text{PC}}, W^{(i)}, z, r^{(i)}; w^{(i)}$) for $i \in \{1, 2\}$ and Plonkish_{Eval}($\text{ck}_{\text{PC}}, \{s_i(X), g_i(X_1, \dots, X_M)\}_{i=1}^{N_g}, \{r_i\}_{i=1}^M; \{w^{(i)}\}_{i=1}^M$)

to recover the aggregated polynomial $F_V(X)$ and the auxiliary polynomial $a^{(i)}(X)$. The consistency check ensures that $a^{(i)}(X)$ correctly encodes the difference between the committed vectors and the wire polynomials $w^{(i)}(X)$.

• **Vector Decomposition:** Subsequently, using transcripts with distinct randomness ρ , $\mathcal{E}$ further decomposes $F_V(X)$ to extract the individual encoding polynomials $F_{L,k}^{(i)}, F_{R,k}^{(i)}, F_{P,k}^{(i)}$.

Finally, $\mathcal{E}$ decodes these polynomials to obtain the vector witnesses (L_k, R_k, P_k) . The validity of these witnesses is

guaranteed because the wire polynomials $w^{(i)}(X)$ (extracted in the first step) are already verified to satisfy the Plonkish circuit constraints, and the soundness of the consistency check implies that the extracted vectors are correctly embedded within these wire polynomials. The full proof is deferred to Section A-D $\square$

V. COUGAR: CUBIC ROOT VERIFIER IPA

A. USING SUBLINEAR VERIFIER PCS

To reduce the verifier complexity, we adopt the sublinear verifier PCS, Leopard_{PC} [40], building upon the previous construction. This approach achieves a cubic root complexity for the verifier. Full details of Leopard_{PC} are provided in Section D.

B. COMPLEXITY ANALYSIS

In this subsection, we provide a complexity analysis of Cougar, broken down into Protocol.Row, Protocol.Col, and AggMEC. Throughout this section, we focus only on the asymptotic complexity. The concrete cost analyses are provided in Section E.

1. Row-reduction, Algorithm 1

- [Prover Cost]: To generate commitments L, R , and $\hat{P}$ in the i -th round, $\mathcal{P}$ performs $O(\frac{N}{2^i}) \mathbb{G}_p$ operations and $O(n \log_2 m) \mathbb{G}_q$ operations. For updating $\hat{G}, \hat{H}$ and $\hat{a}, \hat{b}, \hat{c}$ in the i -th round, $\mathcal{P}$ performs $O(\frac{m}{2^i}) \mathbb{G}_p$ operations and $O(n \cdot \frac{m}{2^i}) \mathbb{Z}_p$ operations, respectively. Consequently, the total prover cost is $O(N) \mathbb{Z}_p$ operations and $O(N) \mathbb{G}_p$ operations.
- [Verifier Cost]: For updating $\hat{G}, \hat{H}$, and $\hat{c}$ in the i -th round, $\mathcal{V}$ performs $O(\frac{m}{2^i}) \mathbb{G}_p$ operations and 2 multiplications in $\mathbb{Z}_p$. Thus, the total verifier cost is $O(m) \mathbb{G}_p$ operations and $O(\log_2 m) \mathbb{Z}_p$ operations.
- [Communication Cost]: In each round, $\mathcal{P}$ sends $L, R, \hat{P}, c_L$, and c_R . The total communication cost is $3 \log_2 m |\mathbb{G}_q| + 2 \log_2 m |\mathbb{Z}_p|$.

2. Column-reduction, Algorithm 2

- [Prover Cost]: For an inner product involving c_L and c_R in the i -th round, the prover performs $O(\frac{n}{2^i}) \mathbb{Z}_p$ operations. For updating $\hat{P}, \hat{a}, \hat{b}$, and $\hat{c}$ in the i -th round, $\mathcal{P}$ performs $O(\frac{n}{2^i}) \mathbb{G}_p$ and $\mathbb{Z}_p$ operations, and $O(\frac{n}{2^i} \log_2 m) \mathbb{G}_q$ operations. Thus, the total prover cost is $O(n \log_2 m) \mathbb{G}_q$ operations.
- [Verifier Cost]: For updating $\hat{c}$ in each round except the final one, $\mathcal{V}$ performs 2 multiplications in $\mathbb{Z}_p$. In the final round, $\mathcal{V}$ performs one $\mathbb{Z}_p$ operation for verification. Thus, the total verifier cost is $O(\log_2 n) \mathbb{Z}_p$ operations.
- [Communication Cost]: In each round, the prover sends $\hat{P}, c_L$, and c_R . The total communication cost is $\log_2 n |\mathbb{G}_q| + 2 \log_2 n |\mathbb{Z}_p|$.

3. Aggregated MEC, Algorithm 3

- [Prover Cost]: From line 1 to 11, $\mathcal{P}$ processes at most $\log_2 N$ polynomials. $\mathcal{P}$ performs $O(n \log_2 N)$

operations across $\mathbb{Z}_p, \mathbb{G}_p$, and $\mathbb{G}_q$. The cost of Eval and Plonkish_{Eval} depends on the degree bound D , denoted as $O(\|\mathcal{P}_{\text{Eval}}(D)\|)$. Thus, the total prover cost is $O(n \log_2 N + \|\mathcal{P}_{\text{Eval}}(D)\|)$.

- [Verifier Cost]: From line 1 to 11, $\mathcal{V}$ performs $O(\log_2 N) \mathbb{G}_q$ operations. The cost of Eval and Plonkish_{Eval} is $O(\|\mathcal{V}_{\text{Eval}}(D)\|)$. Thus, the total verifier cost is $O(\log_2 N + \|\mathcal{V}_{\text{Eval}}(D)\|)$.
- [Communication Cost]: From line 1 to 11, $\mathcal{P}$ sends 4 elements of $\mathbb{G}_q$ and 5 field elements to $\mathcal{V}$. Additionally, for Eval and Plonkish, the prover sends $O(\|\Pi_{\text{Eval}}(D)\|)$. Thus, the total communication cost is $O(\|\Pi_{\text{Eval}}(D)\|)$.

C. ACHIEVING CUBIC ROOT VERIFIER COMPLEXITY

Let $N = mn$ be the length of the witness vectors. We set the parameters as $n = \sqrt[3]{N^2}$ and $m = \sqrt[3]{N}$. Recall that, based on the interpolation strategy defined over the roots of unity, the degree bound of the wire polynomials is $D = O(n \log m)$.

Since Leopard features $(\|\mathcal{P}_{\text{Eval}}(D)\|, \|\mathcal{V}_{\text{Eval}}(D)\|, \|\Pi_{\text{Eval}}(D)\|) = (O(D), O(\sqrt{D}), O(\log_2 D))$, substituting $D = O(n \log m)$ yields the following complexities:

- **Verifier Cost:** Substituting the parameters, the verifier complexity becomes $O(\sqrt{D}) = O(\sqrt{n \log m}) = O(\sqrt{N^{2/3} \log N}) = O(\sqrt[3]{N} \sqrt{\log N})$.
- **Prover Cost:** The dominant cost remains $O(N)$ from the reduction phases, as the $O(D)$ cost from AggMEC is subdominant (since $D \ll N$).
- **Proof Size:** The communication cost comprises $O(\log N)$ from the reduction phases and $O(\log D)$ from AggMEC. Since $\log D = O(\log(n \log m)) \approx O(\log N)$, the total proof size preserves the logarithmic complexity, i.e., $O(\log N)$.

Thus, we conclude that Cougar achieves a verifier complexity proportional to the cubic root of N while maintaining a succinct proof size.

Theorem V.1. *Cougar is an IPA that features $O(\log_2 N)$ communication cost, $O(N)$ prover cost, and $O(\sqrt[3]{N} \sqrt{\log_2 N})$ verifier cost, where N is the length of the witness. Cougar satisfies perfect completeness and computational witness-extended emulation under the DL assumption.*

Proof. The prover, verifier, and communication costs are verified in the analysis above. By combining Theorem IV.1, Theorem IV.2, Theorem IV.3, and the soundness of Leopard under the DL assumption [40], it follows that Cougar satisfies perfect completeness and computational witness-extended emulation under the DL assumption. $\square$

VI. COMPARISON WITH OTHER TRANSPARENT ZK

In this section, we provide comparison among transparent ZK arguments. In Table 3, we present a comparison for asymptotic complexities of transparent ZK arguments, all of which do not require a trusted setup and achieve non-interactivity via the Fiat-Shamir heuristic [28]. The protocols are grouped by their underlying assumptions:

TABLE 3. Comparison for transparent ZK arguments

Scheme	Communication	$\mathcal{P}$'s comp.	$\mathcal{V}$'s comp.	Assump.
Groth, Seo [34], [54]	$O(\sqrt{N})\mathbb{G}_1$	$O(N \log^2 N)m$	$O(N)m$	DL
Bulletproofs [11], [16]	$O(\log N)\mathbb{G}_1$	$O(N)\tau_1$	$O(N)\tau_1$	DL
Spartan [55]	$O(\sqrt{N})\mathbb{G}_1$	$O(N)\tau_1$	$O(\sqrt{N})\tau_1$	DL
Xiphos [45], [57]	$O(\log N)\mathbb{G}_t$	$O(N)\tau_1$	$O(\log N)\tau_t$	SXDH
Protocol2 [41]	$O(\sqrt{\log N})\mathbb{G}_t$	$O(N2^{\sqrt{\log N}})\tau_1$	$O(N)\tau_1$	DL [†] , DPair
Leopard [40], [41]	$O(\log N)\mathbb{G}_t$	$O(N)\tau_1$	$O(\sqrt{N})\tau_2$	DL [†]
Protocol4 [41]	$O(\log N)\mathbb{G}_q$	$O(N)\tau_p$	$O(\sqrt{N} \log N)\tau_q$	DL
Supersonic [19]	$O(\log N)\mathbb{G}_U$	$O(N \log N)u$	$O(\log N)u$	UO
Dew [2]	$O(1)\mathbb{G}_U$	$O(N^2)u$	$O(\log N)u$	UO, GGM
STARK [5]	$O(\log^2 N)\mathbb{H}$	$O(N \log^2 N)h$	$O(\log^2 N)h$	CR hash
Fractal [23]	$O(\log^2 N)\mathbb{H}$	$O(N \log N)h$	$O(\log^2 N)h$	CR hash
Virgo [65]	$O(d \log N)\mathbb{H}$	$O(N \log N)h$	$O(d \log N)h$	CR hash
Brakedown [33]	$O((N^{1/t})\mathbb{H})$	$O(N)h$	$O(N^{1/t})h$	CR hash
Lattice BP [14], [18]	$O(\text{polylog} N)R_q$	$O(N)R$	$O(N)R$	(M)SIS
HSS [37]	$O(\sqrt{N} \text{polylog} N)R_q$	$O(N \text{polylog} N)R$	$O(\sqrt{N} \text{polylog} N)R$	MSIS, MLWE
Greyhound [46]	$O(\text{polylog} N)R_q$	$O(N)R$	$O(\sqrt{N})R$	MSIS
This Work	$O(\log N)\mathbb{G}_t$	$O(N)\tau_1$	$O(\sqrt[3]{N} \sqrt{\log N})\tau_2$	DL [†]

N : circuit size, d : circuit depth, w : input size, $\text{polylog} N$: polylogarithmic function in N
 $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t)$: bilinear groups, $(\mathbb{G}_p, \mathbb{G}_q)$: elliptic curve groups of order p and q , $\mathbb{G}_U$: group of unknown order, $\mathbb{H}$: hash function, R_q : power-of-two cyclotomic ring $\mathbb{Z}_q[x]/(X^N + 1)$,
 m, p, h, τ_i, u, R : operation of field, pairing, hash, $\mathbb{G}_t, \mathbb{G}_U, R_q$
UOGroup: unknown-order group (strong RSA & adaptive root assumptions), CR hash: collision-resistant hashes, DL[†]: DL assumption over pairing-friendly elliptic curves, MSIS: Module-short integer solution, MLWE: Module-learning with error.

A. COMPARISON WITH DL-BASED ZK ARGUMENTS.

DL-based ZK arguments leverage algebraic group structures. Groth [34], [35] and Seo [54] proposed constant-round ZK arguments with sublinear communication. In contrast, Bootle et al. [11], [16] introduced a recursive halving technique that significantly reduces communication to logarithmic size, at the cost of increasing the number of rounds to logarithmic. However, a major limitation of their approach is the linear verifier complexity. To address this issue, several schemes have been proposed [45], [55], [57], either by trading off communication cost or by relying on stronger cryptographic assumptions.

Compared to these works, Cougar relies on only standard assumption, DL assumption and achieve short verifier complexity, cubic root verifier complexity.

B. COMPARISON WITH UO-BASED ZK ARGUMENTS.

To mitigate the high verifier cost inherent in the DL setting, an alternative approach is to construct ZK arguments based on UO groups [2], [19]. Supersonic [19] achieves both logarithmic communication and verifier complexity. Going further, Dew [2] achieves constant-size communication, surpassing the logarithmic bound. However, both schemes incur superlinear prover complexity. Moreover, in practice, the unit operations in the UO setting are typically more expensive than those in the DL setting, so the prover cost is expected to be higher than ours, making these approaches potentially less

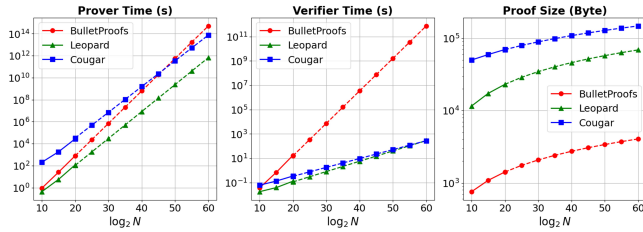
practical.

C. COMPARISON WITH HASH-BASED ZK ARGUMENTS.

Cryptographic groups offer valuable homomorphic structure, enabling modular and composable proof systems. However, group operations are typically more expensive than field operations. Several ZK arguments based on cryptographic hash functions have been proposed [5], [23], [33], [65], achieving linear prover complexity and polylogarithmic verifier time and communication. In contrast, our ZK arguments achieve at most logarithmic communication, which is expected to yield lower concrete communication costs in practice. Moreover, our group-based constructions integrate naturally with cryptographic primitives such as signature schemes [53], [59] or incrementally verifiable computation [17], [43].

D. COMPARISON WITH LATTICE-BASED ZK ARGUMENTS.

To address the lack of quantum resistance in DL-based constructions, several lattice-based ZK arguments have been proposed [14], [18], [37], [46], achieving polylogarithmic communication and sublinear verifier complexity. However, they often suffer from sampling issues and soundness gaps, typically requiring multiple repetitions or carefully chosen parameters to ensure soundness. In contrast, our constructions avoid such issues. Since they rely on cryptographic group settings with inherent soundness guarantees, they do not require repeated executions or delicate parameter tuning, leading to



We present log-log plots of each metric for various values of N ranging from 2^{10} to 2^{60} . Solid lines (from 2^{10} to 2^{20}) represent empirical results, and dotted lines (from 2^{20} to 2^{60}) show estimated values from linear regression based on those results.

FIGURE 3. Evaluation results of each IPA.

more efficient performance in practice.

VII. IMPLEMENTATION

We implemented Cougar with a famous half-pairing cycle of curves: BN254 and Grumpkin. For PCS, we utilize $\text{Leopard}_{\text{PC}}$ instantiated with BN254. We used the Fiat-Shamir transformation [28] to make them non-interactive. Every code was written in Rust, and every experiment was done in the following setting: a single thread of an AMD EPYC 7543P (2.8GHz) CPU with 512GB RAM.

A. EVALUATION RESULTS

We implemented Cougar by following the algorithms described in the paper, with optimization techniques in Section B. For elliptic curve operations in BN254 and Grumpkin, we used `halo2curves` crate [49] of version 0.6.1. In addition, we utilized some parts of `halo2_proofs` crate [50] of version 0.3.0 to deal with polynomials during AggMEC. For more detailed information, we recommend that the reader refer to our source code².

1) Comparison with IPAs based on the DL Assumption.

For comparison, we also implemented BulletProofs (BP_{IP}) [16] and Leopard [40], [41] under our experimental setting and conducted the same experiments as above. To implement BP_{IP} , we chose the `Secp256k1` curve, which is used in many cryptocurrencies. On the other hand, to implement Leopard, we chose the `BLS12-381` curve, which is a well-known pairing-friendly curve. We used `halo2curve` crate of the same version as above and `blstrs` crate [29] of version 0.7.1 for `Secp256k1` and `BLS12-381`, respectively. We note that `Secp256k1`, `BLS12-381`, and `BN254` are known to provide 127-bit, 117-bit, and 102-bit security for the DL assumption, respectively. For implementing Leopard, we followed several optimization tricks introduced by [40].

We report the elapsed time for proof generation and verification for each IPA, along with the corresponding proof sizes. Experiments were conducted for various values of N , ranging from 2^{10} to 2^{20} , and the results are presented and visualized in Table 4. Due to limitations in our environment, we could

TABLE 4. Evaluation results of each IPA for various N from 2^{10} to 2^{20} .

N	Proving time (s)			Verification time (s)			Proof Size (KB)		
	BPs	Leopard	Cougar	BPs	Leopard	Cougar	BPs	Leopard	Cougar
2^{10}	0.91	0.44	206.5	0.04	0.02	0.06	0.76	11.58	50.14
2^{11}	1.73	0.64	221.8	0.07	0.02	0.08	0.82	12.74	52.51
2^{12}	3.37	1.24	452.6	0.11	0.02	0.09	0.89	13.89	54.11
2^{13}	6.59	1.86	854.8	0.20	0.03	0.10	0.95	15.04	55.71
2^{14}	13.03	3.64	902.4	0.38	0.03	0.12	1.02	16.19	58.08
2^{15}	25.83	5.58	1842	0.70	0.04	0.13	1.09	17.34	59.68
2^{16}	51.10	11.25	3315	1.27	0.05	0.15	1.15	18.50	61.28
2^{17}	101.5	18.14	7014	2.48	0.06	0.21	1.22	19.65	64.42
2^{18}	201.8	36.11	13051	4.55	0.08	0.23	1.28	20.80	66.02
2^{19}	401.6	60.12	26646	8.71	0.09	0.30	1.35	21.95	67.62
2^{20}	801.2	120.1	27866	17.34	0.13	0.35	1.42	23.10	69.98

not directly run experiments for larger parameters than 2^{20} . Nevertheless, we estimated results for larger N s, up to 2^{60} , via extrapolation based on linear regression with the data in Table 4. We visualize the estimations in Figure 3.

- **Prover Time:** The performance ranking is Leopard, BP_{IP} , and then Cougar. While all schemes have $O(N)$ prover complexity, BP_{IP} requires updating a linear-sized CRS, whereas the others update only sublinear-sized CRS. Despite this advantage, Cougar performs relatively heavy computations (still sublinear, but with a large constant factor) due to the AggMEC component. As a result, Cougar is slower than BP_{IP} in the range of $N = 2^{10}$ to 2^{45} . After $N = 2^{45}$, Cougar is expected to achieve a shorter prover time.
- **Verifier Time:** The performance ranking is Leopard, Cougar, and then BP_{IP} for $N > 2^{12}$. Thanks to their sublinear verifier complexity, both Leopard and Cougar require fewer computations than BP_{IP} . Although Cougar features cubic-root complexity while Leopard achieves square-root complexity, the implementation results show that Leopard is more efficient. As with the prover, this is because Cougar involves heavier computations due to the AggMEC component. However, as N approaches 2^{60} , we expect that the verifier time of Leopard exceeds that of Cougar due to their asymptotic complexity difference.
- **Proof size:** The performance ranking is BP_{IP} , Leopard, and then Cougar. All three schemes achieve logarithmic proof size, but they rely on different cryptographic groups. Since BP_{IP} uses a pairing-free construction, it can be instantiated with more efficient elliptic curves in practice. In contrast, Leopard and Cougar are constrained by the requirements of pairing-friendly curves. Moreover, since Cougar includes an additional AggMEC proof component, it results in a relatively larger proof size than Leopard.

In contrast to Bulletproofs and Leopard, one can figure out that the proving time of Cougar increases stepwise. This is because the proving time of Leopard highly depends on D . As shown in Table 6 (concrete parameter selection), the rate of the increase in D is slower than N , and more importantly, this tendency exactly coincides with the tendency of the proof generation time depicted in Figure 3.

²<https://github.com/Cryptology-Algorithm-Lab/Cougar>

TABLE 5. Evaluation results of HyraxPCS. $\mathcal{P}$, $\mathcal{V}$, $\mathcal{C}$, refers to proving time, verification time, and communication cost, respectively.

N	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}	2^{17}	2^{18}	2^{19}	2^{20}
$\mathcal{P}$ (s)	0.057	0.093	0.141	0.271	0.465	0.845	1.420	2.771	4.747	9.323	17.117
$\mathcal{V}$ (s)	0.007	0.009	0.007	0.013	0.012	0.018	0.020	0.032	0.036	0.049	0.057
$\mathcal{C}$ (KB)	1.45	2.48	2.54	4.61	4.67	8.79	8.86	17.11	17.17	33.67	33.74

2) Comparison with Hyrax as a PCS.

Recall that one of the main uses of IPA is to instantiate PIOPs, and the Cougar also can be naturally utilized as a PCS as well. For this reason, we also compare the PCS version of Cougar with HyraxPCS [60], which is another PCS based on the DL assumption and featuring a squared root verifier and communication costs. Although both BulletProofs and Leopard can be converted to PCS, we will not compare Hyrax – PCS with them because they have already been compared with ours in terms of IPA.

Since HyraxPCS does not require pairing operations, we used the Secp256k1 curve to implement it. We conducted the same experiments and `halo2curves` library as above under the same environment. The results are given in Table 5. We can observe that the verifier time of HyraxPCS is faster than that of Cougar, even faster than that of Leopard, in our experiment. Moreover, the communication cost is lower in all experimental settings. This is because HyraxPCS does not rely on pairing operations, so they do not require sending heavy group elements in $\mathbb{G}_T$. Nevertheless, we note that HyraxPCS features square root communication cost, so the communication cost increases extremely fast when N becomes large. According to our experimental result, the communication cost of HyraxPCS surpasses that of Leopard when $N \geq 2^{19}$, and the (expected) crossing over point between HyraxPCS and Cougar is $N = 2^{23}$.

In addition, we also estimate the verification cost by conducting linear regression on the data in Table 5, as we did for comparing IPAs. With the same notation as the previous paragraph, we obtain the regression coefficients $(\hat{\alpha}, \hat{\beta}) = (0.325, -10.650)$. Since the regression coefficient for Cougar is $(0.243, -6.465)$, i.e., having a smaller $\hat{\alpha}$ than HyraxPCS, we can conclude that the verification time of the proposed Cougar is asymptotically faster than that of HyraxPCS.

B. EFFICIENCY ANALYSIS

We implemented Cougar in Rust, employing the BN254 and Grumpkin curves as a half-pairing cycles. We run Cougar from $N = 2^{10}$ to $N = 2^{20}$ and compared it with previous IPAs from the DL assumption, including Bulletproofs and Leopard. Under our experimental results, the measured time spent by verifier in $N = 2^{20}$ is 0.346s, which is $50\times$ speed-up from Bulletproofs. We also emphasize that as N enlarges, the growth of the verifier time of Cougar is much slower than that of Bulletproofs and Leopard. In addition, we checked that the (estimated) verifier time of Cougar becomes faster than that of Leopard for a larger N , though the concrete verifier time of Cougar is slower than Leopard in our experiments. For more details about implementation, we recommend the reader refer

to our source code in github³.

VIII. CONCLUSION

In this work, we presented **Cougar**, the first inner product argument (IPA) to achieve **cubic-root verifier complexity** while maintaining a **logarithmic proof size** under the discrete logarithm assumption with a transparent setup. Our construction synergizes the strengths of existing square-root verifier IPAs, such as **Leopard**, by combining them with a generalized two-tier commitment framework. A key technical novelty lies in our disjoint encoding strategy, which efficiently maps witness vectors to non-overlapping polynomial domains. This approach is crucial for enabling the efficient consistency checks required for our cubic-root verification structure without compromising the compact proof size. Through comprehensive experiments, we demonstrated that the verification cost of **Cougar** increases much slower than two previous IPAs, BulletProofs and Leopard, under the same DL assumption and transparent setup.

We identify several avenues for future work to further improve **Cougar**. First, while **Cougar** improves asymptotic complexity, it currently exhibits lower concrete efficiency for practical instance sizes (N) compared to square-root verifiers. Our regression analysis suggests that **Cougar** becomes more efficient than **Leopard** when $\log_2 N > 60$; reducing this crossover point is a challenging yet important open problem.

Second, the reliance on **Plonkish** arithmetization limits the maximum circuit size to the domain size of the underlying elliptic curve (e.g., 2^{28} in BN254). To overcome this scalability bottleneck, future iterations could employ interpolation-free constraint systems, such as SuperSpartan [58] or HyperPlonk [22], which do not rely on Lagrange interpolation.

Finally, extending **Cougar** to support recursive composition poses a non-trivial challenge. Unlike standard homomorphic commitments, our two-tier commitment structure does not naturally preserve the additive homomorphism required for efficient recursive folding mechanisms (e.g., Halo [15]). We leave the exploration of recursive variants of **Cougar** as an interesting direction for future research.

REFERENCES

- [1] M. Abe, G. Fuchsbaauer, J. Groth, K. Haralambiev, and M. Ohkubo. Structure-preserving signatures and commitments to group elements. *J. Cryptol.*, 29(2):363–421, 2016.
- [2] A. Arun, C. Ganesh, S. V. Lokam, T. Mopuri, and S. Sridhar. Dew: Transparent constant-sized zkSNARKs. In *Public-Key Cryptography – PKC 2023*, pages 542–571. Springer, 2023.
- [3] C. Baum, J. Bootle, A. Cerulli, R. del Pino, J. Groth, and V. Lyubashevsky. Sub-linear lattice-based zero-knowledge arguments for arithmetic circuits. In *Advances in Cryptology – CRYPTO 2018*, pages 669–699. Springer, 2018.
- [4] C. Baum and A. Nof. Concretely-efficient zero-knowledge arguments for arithmetic circuits and their application to lattice-based cryptography. In *Public-Key Cryptography – PKC 2020*, pages 495–526. Springer, 2020.
- [5] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, Report 2018/046, 2018. <https://eprint.iacr.org/2018/046>.

³<https://github.com/Cryptology-Algorithm-Lab/Cougar>

- [6] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable zero knowledge with no trusted setup. In *Annual International Cryptology Conference*, pages 701–732. Springer, 2019.
- [7] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza. Snarks for c: Verifying program executions succinctly and in zero knowledge. In *CRYPTO 2013*, volume 8043 of *LNCS*, pages 90–108. Springer, 2013.
- [8] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward. Aurora: Transparent succinct arguments for r1cs. In *EUROCRYPT 2019*, volume 11476 of *LNCS*, pages 103–128. Springer, 2019.
- [9] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. Succinct non-interactive zero knowledge for a von Neumann architecture. In *USENIX Security 2014*, pages 781–796, 2014.
- [10] D. Boneh, J. Drake, B. Fisch, and A. Gabizon. Efficient polynomial commitment schemes for multiple points and polynomials. *Cryptology ePrint Archive*, 2020.
- [11] J. Bootle, A. Cerulli, P. Chaidos, J. Groth, and C. Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *EUROCRYPT 2016*, volume 9666 of *LNCS*, pages 327–357. Springer, 2016.
- [12] J. Bootle, A. Cerulli, J. Groth, S. Jakobsen, and M. Maller. Arya: Nearly linear-time zero-knowledge proofs for correct program execution. In *ASIACRYPT 2018, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part I*, volume 11272 of *LNCS*, pages 595–626. Springer, 2018.
- [13] J. Bootle, A. Chiesa, and J. Groth. Linear-time arguments with sublinear verification from tensor codes. In *Theory of Cryptography: 18th International Conference, TCC 2020, Durham, NC, USA, November 16–19, 2020, Proceedings, Part II* 18, pages 19–46. Springer, 2020.
- [14] J. Bootle, V. Lyubashevsky, N. K. Nguyen, and G. Seiler. A non-pec approach to succinct quantum-safe zero-knowledge. In *Advances in Cryptology – CRYPTO 2020*, pages 441–469. Springer, 2020.
- [15] S. Bowe, J. Grigg, and D. Hopwood. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive*, Report 2019/1021, 2019.
- [16] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE Symposium on Security and Privacy 2018*, pages 315–334. IEEE Computer Society, 2018.
- [17] B. Bünz, A. Chiesa, W. Lin, P. Mishra, and N. Spooner. Proof-carrying data without succinct arguments. In *Annual International Cryptology Conference*, pages 681–710. Springer, 2021.
- [18] B. Bünz and B. Fisch. Multilinear schwartz-zippel mod n and lattice-based succinct arguments. In *Theory of Cryptography Conference*, pages 394–423. Springer, 2023.
- [19] B. Bünz, B. Fisch, and A. Szepieniec. Transparent snarks from dark compilers. In *EUROCRYPT 2020, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*, volume 12105 of *LNCS*, pages 677–706. Springer, 2020.
- [20] B. Bünz, M. Maller, P. Mishra, N. Tyagi, and P. Vesely. Proofs for inner pairing products and applications. In *ASIACRYPT 2021, Singapore, December 6-10, 2021, Proceedings, Part III*, volume 13092 of *LNCS*, pages 65–97. Springer, 2021.
- [21] M. Burmester, Y. Desmedt, and T. Beth. Efficient zero-knowledge identification scheme for smart cards. *Comput. J.*, 35(1):21–29, 1992.
- [22] B. Chen, B. Bünz, D. Boneh, and Z. Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 499–530. Springer, 2023.
- [23] A. Chiesa, D. Ojha, and N. Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. In *EUROCRYPT 2020*, volume 12105 of *LNCS*, pages 769–793. Springer, 2020.
- [24] H. Chung, K. Han, C. Ju, M. Kim, and J. H. Seo. Bulletproofs+: Shorter proofs for a privacy-enhanced distributed ledger. *IEEE Access*, 10:42067–42082, 2022.
- [25] V. Daza, C. Ràfols, and A. Zacharakis. Updateable inner product argument with logarithmic verifier and applications. In *PKC 2020*, volume 12110 of *LNCS*, pages 527–557. Springer, 2020.
- [26] L. Eagen, S. Kanjalkar, T. Ruffing, and J. Nick. Bulletproofs++: Next generation confidential transactions via reciprocal set membership arguments. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 249–279. Springer, 2024.
- [27] U. Feige, A. Fiat, and A. Shamir. Zero-knowledge proofs of identity. *J. Cryptol.*, 1(2):77–94, 1988.
- [28] A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO 1986*, volume 263 of *LNCS*, pages 186–194. Springer, 1987.
- [29] Filecoin. blstrs, 2024. <https://github.com/filecoin-project/blstrs>.
- [30] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, Report 2019/953, 2019. <https://eprint.iacr.org/2019/953.pdf>.
- [31] S. Gao, Z. Peng, F. Tan, Y. Zheng, and B. Xiao. Symmeproof: Compact zero-knowledge argument for blockchain confidential transactions. *IEEE Transactions on Dependable and Secure Computing*, 20(3):2289–2301, 2023.
- [32] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM J. Comput.*, 18(1):186–208, 1989.
- [33] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby. Brakedown: Linear-time and field-agnostic snarks for r1cs. In *Annual International Cryptology Conference*, pages 193–226. Springer, 2023.
- [34] J. Groth. Linear algebra with sub-linear zero-knowledge arguments. In *CRYPTO 2009*, volume 5677 of *LNCS*, pages 192–208. Springer, 2009.
- [35] J. Groth. Efficient zero-knowledge arguments from two-tiered homomorphic commitments. In *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 431–448. Springer, 2011.
- [36] J. Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT 2016*, volume 9666 of *LNCS*, pages 305–326. Springer, 2016.
- [37] I. Hwang, J. Seo, and Y. Song. Concretely efficient lattice-based polynomial commitment from standard assumptions. In *Annual International Cryptology Conference*, pages 414–448. Springer, 2024.
- [38] A. Jivanyan. Lelantus: Towards confidentiality and anonymity of blockchain transactions from standard assumptions. *Cryptology ePrint Archive*, Report 2019/373, 2019.
- [39] A. Kate, G. M. Zaverucha, and I. Goldberg. Constant-size commitments to polynomials and their applications. In *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, 2010.
- [40] S. Kim, G. Lee, H. Lee, and J. H. Seo. Leopard: Sublinear verifier inner product argument under discrete logarithm assumption. *IEEE Transactions on Information Forensics and Security*, 18:5332–5344, 2023.
- [41] S. Kim, H. Lee, and J. H. Seo. Efficient zero-knowledge arguments in discrete logarithm setting: Sublogarithmic proof or sublinear verifier. In *ASIACRYPT 2022, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part II*, volume 13792 of *LNCS*, pages 403–433. Springer, 2022.
- [42] A. Kothapalli and B. Parno. Algebraic reductions of knowledge. In *Annual International Cryptology Conference*, pages 669–701, 2023.
- [43] A. Kothapalli, S. Setty, and I. Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.
- [44] H. Lee and J. H. Seo. TENET: sublogarithmic proof and sublinear verifier inner product argument without a trusted setup. In *Advances in Information and Computer Security - 18th International Workshop on Security, IWSEC 2023, Yokohama, Japan, August 29-31, 2023, Proceedings*, volume 14128 of *Lecture Notes in Computer Science*, pages 214–234. Springer, 2023.
- [45] J. Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In *TCC 2021, Raleigh, NC, USA, November 8-11, 2021, Proceedings, Part II*, volume 13043 of *LNCS*, pages 1–34. Springer, 2021.
- [46] N. K. Nguyen and G. Seiler. Greyhound: Fast polynomial commitments from lattices. In *Annual International Cryptology Conference*, pages 243–275. Springer, 2024.
- [47] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO 1991*, volume 576 of *LNCS*, pages 129–140. Springer, 1991.
- [48] N. Pippenger. On the evaluation of powers and monomials. *SIAM Journal on Computing*, 9(2):230–250, 1980.
- [49] PSE. halo2curves, 2024. <https://github.com/privacy-scaling-explorations/halo2curves>.
- [50] PSE. halo2_proofs, 2024. https://github.com/zcash/halo2/tree/main/halo2_proofs.
- [51] J. Renes, C. Costello, and L. Batina. Complete addition formulas for prime order elliptic curves. In M. Fischlin and J. Coron, editors, *EUROCRYPT 2016*, volume 9665 of *LNCS*, pages 403–428. Springer, 2016.
- [52] E. B. Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *IEEE Symposium on Security and Privacy 2014*, pages 459–474. IEEE, 2014.
- [53] C.-P. Schnorr. Efficient identification and signatures for smart cards. In *Conference on the Theory and Application of Cryptology*, pages 239–252. Springer, 1989.

- [54] J. H. Seo. Round-efficient sub-linear zero-knowledge arguments for linear algebra. In *PKC 2011*, volume 6571 of *LNCS*, pages 387–402. Springer, 2011.
- [55] S. Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *CRYPTO 2020*, volume 12172 of *LNCS*, pages 704–737. Springer, 2020.
- [56] S. Setty, S. Angel, T. Gupta, and J. Lee. Proving the correct execution of concurrent services in zero-knowledge. In *OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, pages 339–356. USENIX Association, 2018.
- [57] S. Setty and J. Lee. Quarks: Quadruple-efficient transparent zkSNARKs. *Cryptology ePrint Archive*, 2020.
- [58] S. Setty, J. Thaler, and R. Wahby. Customizable constraint systems for succinct arguments. *Cryptology ePrint Archive*, 2023.
- [59] S.-F. Sun, M. Au, J. Liu, and T. Yuen. Ringet 2.0: A compact accumulator-based (linkable ring signature) protocol for blockchain cryptocurrency monero. In *ESORICS 2017*, volume 10493 of *LNCS*, pages 456–474. Springer, 2017.
- [60] R. S. Wahby, I. Tzialis, A. Shelat, J. Thaler, and M. Walfish. Doubly-efficient zkSNARKs without trusted setup. In *IEEE Symposium on Security and Privacy 2018*, pages 926–943. IEEE, 2018.
- [61] T. Xie, Y. Zhang, and D. Song. Orion: Zero knowledge proof with linear prover time. In *Annual International Cryptology Conference*, pages 299–328. Springer, 2022.
- [62] T. H. Yuen, S. Sun, J. K. Liu, M. H. Au, M. F. Esgin, Q. Zhang, and D. Gu. Ringet 3.0 for blockchain confidential transaction: Shorter size and stronger security. In *Financial Cryptography and Data Security 2020*, volume 12059 of *LNCS*, pages 464–483. Springer, 2020.
- [63] A. Zapico, V. Buterin, D. Khovratovich, M. Maller, A. Nitulescu, and M. Simkin. Caulk: Lookup arguments in sublinear time. In *CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 3121–3134. ACM, 2022.
- [64] zcash. The halo2 book, 2022. <https://zcash.github.io/halo2/>.
- [65] J. Zhang, T. Xie, Y. Zhang, and D. Song. Transparent polynomial delegation and its applications to zero knowledge proof. In *IEEE Symposium on Security and Privacy 2020*, pages 859–876. IEEE, 2020.

APPENDIX A DEFERRED PROOFS

A. GENERAL FORKING LEMMA

The security of our reduction protocols relies on the ability to extract a witness by rewinding the adversary. To formalize this extraction procedure, we utilize the *General Forking Lemma* [11], which provides a probabilistic bound for extracting a witness from a polynomially bounded tree of accepting transcripts. The formal statement is as follows:

Theorem A.1 (General Forking Lemma [11]). *Let $(\mathcal{K}, \mathcal{P}, \mathcal{V})$ be a $(2\mu + 1)$ -move, public coin interactive protocol with μ challenges $x_1, \dots, x_\mu$ in sequence. Let $n_i \geq 1$ for $i \in [\mu]$. Consider an $(n_1, \dots, n_\mu)$ -tree of accepting transcripts with challenges in the following format: The tree has $N = \prod_{i=1}^{\mu} n_i$ leaves with depth μ . The root of the tree is labeled with the statement. Each node of depth i has exactly n_i children, each labeled with a distinct value of the i -th challenge x_i .*

Let $\mathcal{E}$ be a witness extractor that succeeds with probability $1 - \text{negl}(\lambda)$ for some negligible function $\text{negl}(\lambda)$ in extracting a witness from an $(n_1, \dots, n_\mu)$ -tree of accepting transcripts in probabilistic polynomial time. Assume that $\prod_{i=1}^{\mu} n_i$ is bounded above by a polynomial in the security parameter λ . Then, $(\mathcal{K}, \mathcal{P}, \mathcal{V})$ has witness-extended emulation.

B. SECURITY PROOF OF PROTOCOL.ROW

Proof. Completeness. For the base case $m = 1$, completeness follows directly from the completeness of Protocol.Col and AggMEC. For $m > 1$, we show that if the input $(\mathbf{G}, \mathbf{H}, \text{ck}_{P,s}, \mathbf{P}, c; \mathbf{a}, \mathbf{b}) \in \mathcal{R}_{\text{TC}}^{m,n}$, then the updated input

$(\widehat{\mathbf{G}}, \widehat{\mathbf{H}}, \text{ck}_{P,s+1}, \widehat{\mathbf{P}}, \widehat{c}; \widehat{\mathbf{a}}, \widehat{\mathbf{b}}) \in \mathcal{R}_{\text{TC}}^{m/2,n}$. From the protocol description, the prover computes $\widehat{\mathbf{a}} = \mathbf{a}_L + x\mathbf{a}_R$ and $\widehat{\mathbf{b}} = \mathbf{b}_L + x^{-1}\mathbf{b}_R$. The updated inner product relation holds because:

$$\begin{aligned} \langle \widehat{\mathbf{a}}, \widehat{\mathbf{b}} \rangle &= \langle \mathbf{a}_L + x\mathbf{a}_R, \mathbf{b}_L + x^{-1}\mathbf{b}_R \rangle \\ &= \langle \mathbf{a}_L, \mathbf{b}_L \rangle + x^{-1}\langle \mathbf{a}_L, \mathbf{b}_R \rangle + x\langle \mathbf{a}_R, \mathbf{b}_L \rangle + \langle \mathbf{a}_R, \mathbf{b}_R \rangle \\ &= c + x^{-1}c_L + xc_R = \widehat{c}. \end{aligned}$$

Similarly, the group element $\widehat{\mathbf{P}}$ is correctly formed as a commitment to $\widehat{\mathbf{a}} \parallel \widehat{\mathbf{b}}$ under the updated basis $\widehat{\mathbf{G}} \parallel \widehat{\mathbf{H}}$. Thus, the reduction step preserves the relation.

Witness-Extended Emulation. We construct an expected polynomial-time extractor $\mathcal{E}_{\text{Row}}$ that extracts a valid witness $(\mathbf{a}, \mathbf{b})$ from the tree of accepting transcripts. We use the General Forking Lemma Theorem A.1.

Base Case ($m = 1$): The extractor $\mathcal{E}_{\text{Row}}$ simply invokes $\mathcal{E}_{\text{Col}}$ to extract the witness $(\mathbf{a}, \mathbf{b})$ for the column reduction phase. Since $\mathcal{E}_{\text{Col}}$ has witness-extended emulation, it returns a valid witness with overwhelming probability.

Inductive Step ($m > 1$): Assume there exists an extractor for $m/2$. We construct an extractor for m . The extractor $\mathcal{E}_{\text{Row}}$ first runs the protocol to obtain a transcript. By rewinding the adversary, it collects 4 accepting transcripts with distinct challenges x_1, x_2, x_3, x_4 for the same initial message $(\mathbf{L}, \mathbf{R}, c_L, c_R)$. Let $(\widehat{\mathbf{a}}_i, \widehat{\mathbf{b}}_i)$ be the witnesses extracted recursively for the i -th branch corresponding to challenge x_i . From the structure of the protocol, the updated commitments satisfy:

$$\widehat{\mathbf{P}}_i = x_i^{-1}\mathbf{L} + \mathbf{P} + x_i\mathbf{R} = [\widehat{\mathbf{a}}_i]\widehat{\mathbf{G}}_i \parallel [\widehat{\mathbf{b}}_i]\widehat{\mathbf{H}}_i, \quad (6)$$

where $\widehat{\mathbf{G}}_i = \mathbf{G}_L + x_i^{-1}\mathbf{G}_R$ and $\widehat{\mathbf{H}}_i = \mathbf{H}_L + x_i\mathbf{H}_R$.

We can rewrite the RHS of Eq. (6) in terms of the original basis $\mathbf{G}_L, \mathbf{G}_R, \mathbf{H}_L, \mathbf{H}_R$:

$$\begin{aligned} [\widehat{\mathbf{a}}_i]\widehat{\mathbf{G}}_i &= [\widehat{\mathbf{a}}_i]\mathbf{G}_L + [x_i^{-1}\widehat{\mathbf{a}}_i]\mathbf{G}_R, \\ [\widehat{\mathbf{b}}_i]\widehat{\mathbf{H}}_i &= [\widehat{\mathbf{b}}_i]\mathbf{H}_L + [x_i\widehat{\mathbf{b}}_i]\mathbf{H}_R. \end{aligned}$$

Since $\mathbf{L}, \mathbf{P}, \mathbf{R}$ are commitments in $\mathbb{G}^{2n}$, let their discrete logarithm representations with respect to the basis $(\mathbf{G}_L, \mathbf{G}_R, \mathbf{H}_L, \mathbf{H}_R)$ be denoted as vectors $\mathbf{l}, \mathbf{p}, \mathbf{r}$. For instance, $\mathbf{P} = [\mathbf{p}^{(1)}]\mathbf{G}_L + [\mathbf{p}^{(2)}]\mathbf{G}_R + [\mathbf{p}^{(3)}]\mathbf{H}_L + [\mathbf{p}^{(4)}]\mathbf{H}_R$. Equation (6) implies the following linear relations in the exponent (assuming DL hardness):

$$x_i^{-1}\mathbf{l}^{(1)} + \mathbf{p}^{(1)} + x_i\mathbf{r}^{(1)} = \widehat{\mathbf{a}}_i \quad (7)$$

$$x_i^{-1}\mathbf{l}^{(2)} + \mathbf{p}^{(2)} + x_i\mathbf{r}^{(2)} = x_i^{-1}\widehat{\mathbf{a}}_i \quad (8)$$

$$x_i^{-1}\mathbf{l}^{(3)} + \mathbf{p}^{(3)} + x_i\mathbf{r}^{(3)} = \widehat{\mathbf{b}}_i \quad (9)$$

$$x_i^{-1}\mathbf{l}^{(4)} + \mathbf{p}^{(4)} + x_i\mathbf{r}^{(4)} = x_i\widehat{\mathbf{b}}_i \quad (10)$$

If these relations did not hold, we could subtract the RHS from the LHS to find a non-trivial linear combination of the basis elements $(\mathbf{G}_L, \dots, \mathbf{H}_R)$ that sums to zero. This would allow us to compute discrete logarithms, contradicting the DL assumption.

Now, we solve for the witness components. From Eq. (7) and Eq. (8), we have:

$$\begin{aligned}\hat{a}_i &= x_i^{-1}l^{(1)} + p^{(1)} + x_i r^{(1)} \\ \hat{a}_i &= l^{(2)} + x_i p^{(2)} + x_i^2 r^{(2)}\end{aligned}$$

Comparing the coefficients of the polynomial in x_i , valid for distinct x_i 's (via Vandermonde argument), we deduce:

$$l^{(1)} = 0, \quad p^{(1)} = l^{(2)}, \quad r^{(1)} = p^{(2)}, \quad r^{(2)} = 0.$$

Let us define $a_L := p^{(1)}$ and $a_R := p^{(2)}$. Then, $\hat{a}_i = a_L + x_i a_R$, which matches the protocol construction.

Similarly, from Eq. (9) and Eq. (10):

$$l^{(4)} = 0, \quad p^{(4)} = l^{(3)}, \quad r^{(4)} = p^{(3)}, \quad r^{(3)} = 0.$$

Defining $b_L := p^{(3)}$ and $b_R := p^{(4)}$, we get $\hat{b}_i = b_L + x_i^{-1} b_R$.

Finally, we verify the inner product relation. The verifier checks:

$$\hat{c}_i = x_i^{-1} c_L + c + x_i c_R.$$

From the extracted witnesses, we know $\hat{c}_i = \langle \hat{a}_i, \hat{b}_i \rangle$. Expanding this:

$$\begin{aligned}\langle \hat{a}_i, \hat{b}_i \rangle &= \langle a_L + x_i a_R, b_L + x_i^{-1} b_R \rangle \\ &= \langle a_L, b_L \rangle + x_i^{-1} \langle a_L, b_R \rangle + x_i \langle a_R, b_L \rangle + \langle a_R, b_R \rangle.\end{aligned}$$

Comparing coefficients of x_i^{-1} , 1 , x_i for 3 distinct challenges yields:

$$c_L = \langle a_L, b_R \rangle, \quad c = \langle a_L, b_L \rangle + \langle a_R, b_R \rangle, \quad c_R = \langle a_R, b_L \rangle.$$

Thus, the extracted vector $(a_L \parallel a_R)$ and $(b_L \parallel b_R)$ is a valid witness for the relation $\mathcal{R}_{TC}^{m,n}$ satisfying the inner product c . By the General Forking Lemma, $\mathcal{E}_{Row}$ runs in expected polynomial time. $\square$

C. SECURITY PROOF OF PROTOCOL.COL

Proof. Completeness. For the base case $n = 1$, completeness follows directly from the completeness of AggMEC and the fact that the initial input is in $\mathcal{R}_{TC}^{1,1}$. For $n > 1$, we show that if the input $(G, H, ck_{p,\mu+s}, P, c; a, b) \in \mathcal{R}_{TC}$, then the updated input $(G, H, ck_{p,\mu+s+1}, \hat{P}, \hat{c}; \hat{a}, \hat{b}) \in \mathcal{R}_{TC}^{1,n/2}$. The prover computes $\hat{a} = a_L + x a_R$ and $\hat{b} = b_L + x^{-1} b_R$. The inner product relation holds as shown in the proof of Theorem IV.1. For the commitment $\hat{P}$, the protocol constructs the vector commitment $\hat{P}$ as:

$$\begin{aligned}\hat{P} &= (P^{(q_1)} \parallel [x]P^{(q_4)}) + (P^{(q_2)} \parallel [x^{-1}]P^{(q_3)}) \\ &= ([a_L]G + [x a_R]G) \parallel ([x^{-1} b_R]H + [b_L]H) \\ &= [\hat{a}]G \parallel [\hat{b}]H.\end{aligned}$$

Since $\hat{P} = \text{Com}_2(ck_{p,\nu}, \hat{P})$, the updated tuple constitutes a valid instance of $\mathcal{R}_{TC}^{1,n/2}$.

Witness-Extended Emulation. We construct an expected polynomial-time extractor $\mathcal{E}_{Col}$ analogous to $\mathcal{E}_{Row}$. The extractor uses a $(\underbrace{3, \dots, 3}_{\log_2 n})$ -tree of accepting transcripts. The

number of leaves $3^{\log_2 n}$ is polynomially bounded, allowing the application of the General Forking Lemma.

Base Case ($n = 1$): The prover sends witnesses a, b directly. $\mathcal{E}_{Col}$ simply reads them from the transcript, which are valid by the soundness of the base check.

Inductive Step ($n > 1$): Assume we can extract witnesses for size $n/2$. We construct an extractor for size n . $\mathcal{E}_{Col}$ collects 3 accepting transcripts with distinct challenges x_1, x_2, x_3 . Let $(\hat{a}_i, \hat{b}_i)$ be the extracted witnesses for the i -th branch. The verification equation implies that the updated commitment chunks $\hat{P}_i$ satisfy linear relations with the underlying chunks $P^{(q_1)}, \dots, P^{(q_4)}$. Specifically, using just 2 distinct challenges (say x_1, x_2), we can set up the following linear systems:

$$\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \end{bmatrix} \begin{bmatrix} P^{(q_1)} \\ P^{(q_2)} \end{bmatrix} = \begin{bmatrix} [\hat{a}_1]G \\ [\hat{a}_2]G \end{bmatrix}, \quad \begin{bmatrix} 1 & x_1^{-1} \\ 1 & x_2^{-1} \end{bmatrix} \begin{bmatrix} P^{(q_3)} \\ P^{(q_4)} \end{bmatrix} = \begin{bmatrix} [\hat{b}_1]H \\ [\hat{b}_2]H \end{bmatrix}. \quad (11)$$

Since $x_1 \neq x_2$, the Vandermonde-style matrices are invertible. By the DL assumption, $\mathcal{E}_{Col}$ can solve these systems to uniquely determine the representations (witnesses) $p^{(q_1)}, \dots, p^{(q_4)}$ such that $P^{(q_1)} = [p^{(q_1)}]G$, etc. Let us define the extracted vectors as $a_L = p^{(q_1)}$, $a_R = p^{(q_2)}$, $b_L = p^{(q_3)}$, and $b_R = p^{(q_4)}$.

Finally, to verify the inner product relation $c = \langle a, b \rangle$, we utilize all 3 challenges. The updated claim $\hat{c}(x) = x^{-1} \langle a_L, b_R \rangle + c + x \langle a_R, b_L \rangle$ is a Laurent polynomial of degree 1 (with x^{-1}). Since the equality holds for 3 distinct points, the constant term must match $c = \langle a_L, b_L \rangle + \langle a_R, b_R \rangle = \langle a, b \rangle$. Thus, the extracted witness (a, b) is valid for $\mathcal{R}_{TC}^{1,n}$. $\square$

D. SECURITY PROOF OF AGGMEC

Proof. (Completeness) Assume that the input tuple $(ck_k, (L_k, R_k, P_k, x_k); (L_k, R_k, P_k))$ satisfies the relations defined in Eq. (3), (4), and (5). By the homomorphic property of the underlying polynomial commitment scheme and the perfect completeness of both Eval and Plonkish_{Eval}, the verifier $\mathcal{V}$ accepts the proofs generated by Eval and Plonkish_{Eval} with probability 1. This establishes the perfect completeness of AggMEC.

(Witness-Extended Emulation) To demonstrate computational witness-extended emulation, we construct an expected polynomial-time extractor $\mathcal{E}_{MEC}$. The goal of $\mathcal{E}_{MEC}$ is to extract a valid witness by rewinding the prover to generate a polynomially bounded tree of accepting transcripts. $\mathcal{E}_{MEC}$ takes public inputs $(ck_k, L_k, R_k, P_k, x_k)$ and outputs witness vectors (L_k, R_k, P_k) that satisfy Eq. (3), Eq. (4), and Eq. (5).

By the general forking lemma [16], it suffices to show that $\mathcal{E}_{MEC}$ can extract a witness from a suitable tree of accepting transcripts in probabilistic polynomial time. We consider a $(6 \log m + 2 \log n + 2, 5)$ -tree of accepting transcripts. Since the number of leaves in this tree is polynomially bounded, the general forking lemma is applicable.

By hypothesis, there exists a PPT extractor $\mathcal{E}_{Eval}$ for PCS.Eval. Additionally, since the premise implies that Plonkish_{Eval} is an Argument of Knowledge (AoK), there

exists a PPT extractor $\mathcal{E}_{\text{Plonkish}}$ that extracts the wire polynomials $\{w^{(i)}(X)\}_{i=1}^M$. $\mathcal{E}_{\text{MEC}}$ invokes these extractors as subroutines.

First, for each coordinate $i \in \{1, 2\}$, $\mathcal{E}_{\text{MEC}}$ retrieves $F_P(X)$ and $w^{(i)}(X)$ by invoking $\mathcal{E}_{\text{Eval}}$ on input $(\text{ck}_{\text{PC}}, P, z, y)$ and $(\text{ck}_{\text{PC}}, W^{(i)}, z, r^{(i)})$, respectively. Using 5 transcripts with distinct challenges τ , $\mathcal{E}_{\text{MEC}}$ extracts F_V , a polynomial $a^{(i)}(X)$, and quotient polynomials $q^{(i)}(X)$. This is achieved by treating the relation $F_P = F_V + \sum_{i=1}^2 (\tau^i a^{(i)} + \tau^{2+i} q^{(i)})$ as a polynomial in τ of degree 4 and performing interpolation. Observe that $w^{(i)}(X)$, $a^{(i)}(X)$, and $q^{(i)}(X)$ satisfy the condition $a^{(i)}(z) = w^{(i)}(z) - q^{(i)}(z)(z^d - 1)$ for $i \in \{1, 2\}$.

Next, utilizing $6 \log m + 2 \log n + 2$ transcripts with distinct challenges ρ , $\mathcal{E}_{\text{MEC}}$ extracts the polynomials $F_{L,k}^{(i)}, F_{R,k}^{(i)}, F_{P,k}^{(i)}$. This is done by regarding the following relationship as a polynomial in ρ of degree $6 \log m + 2 \log n + 1$:

$$F_V = \sum_{i=1}^2 \left(\sum_{k=1}^{\mu} (\rho^{4k-2-i} F_{L,k}^{(i)} + \rho^{4k-i} F_{R,k}^{(i)}) + \rho^{4\mu} \sum_{k=0}^{\mu+\nu} \rho^{2k-1+i} F_{P,k}^{(i)} \right).$$

The extracted polynomials satisfy the commitment relations:

$$\begin{aligned} L_k^{(i)} &= \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, F_{L,k}^{(i)}), \\ R_k^{(i)} &= \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, F_{R,k}^{(i)}), \\ P_k^{(i)} &= \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, F_{P,k}^{(i)}). \end{aligned} \quad (12)$$

Finally, $\mathcal{E}_{\text{MEC}}$ outputs the vectors $\mathbf{L}_k^{(i)}, \mathbf{R}_k^{(i)}, \mathbf{P}_k^{(i)}$ by decoding the polynomials $F_{L,k}^{(i)}, F_{R,k}^{(i)}, F_{P,k}^{(i)}$, respectively.

It remains to verify that these extracted vectors constitute a valid witness satisfying all relations from Eq. (3) to Eq. (5). First, the extraction process ensures that $\mathbf{L}_k^{(i)}, \mathbf{R}_k^{(i)}, \mathbf{P}_k^{(i)}$ satisfy Eq. (12), and consequently Eq. (3). In addition, by the construction of $A^{(i)}$, the polynomial $a^{(i)}$ is identical to the sum of the encoded polynomials, i.e., $a^{(i)} = \sum_{k=1}^{\mu} (F_{L,k}^{(i)} + F_{R,k}^{(i)}) + \sum_{k=0}^{\mu+\nu} F_{P,k}^{(i)}$. This implies that the evaluations of the wire polynomial $w^{(i)}$ at the appropriate domain points ξ^i embed the values of $F_{L,k}^{(i)}, F_{R,k}^{(i)}, F_{P,k}^{(i)}$. Specifically, for each $i \in \{1, 2\}$, the values correspond precisely to the components of $\mathbf{L}_k^{(i)}, \mathbf{R}_k^{(i)}, \mathbf{P}_k^{(i)}$ at the assigned positions. Furthermore, the validity of the wire polynomials $\{w^{(i)}(X)\}_{i=1}^M$, guaranteed by $\mathcal{E}_{\text{Plonkish}}$, implies that $\mathbf{L}_k^{(i)}, \mathbf{R}_k^{(i)}, \mathbf{P}_k^{(i)}$ satisfy the structural relations in Eq. (4) and Eq. (5).

In conclusion, the extracted vectors are valid witnesses, thereby proving that AggMEC satisfies computational witness-extended emulation. $\square$

APPENDIX B OPTIMIZATION

A. CONCRETE PARAMETER SELECTION

Under the suggested parameter in Section V, the verifier cost of Cougar becomes cubic root with respect to the length of the witness vectors. However, when we consider the efficiency on concrete parameters, the huge constant in front of $D = O(n \log_2 m)$ leads to inefficiency, which requires the proper choice of m and n parameters to reduce D . To give a way

to select m and n , we provide a concrete size of D , which is determined by m , n , and q .

$$D = \underbrace{12n \log_2 m \log_2 q + 4n \log_2 q}_{\text{Protocol.Row}} + \underbrace{(4n - 4) \log_2 q}_{\text{Protocol.Col}}. \quad (13)$$

That is, for small N , the term $\log_2 q$ would dominate the verifier's computation cost, so the parameters (m, n) should be selected carefully.

To this end, first recall that the verifier's computational costs consist of (1) checking $c = a \cdot b$ in the last of Protocol.Col through $2 \log_2 N$ field operations, (2) computing $A^{(1)}, A^{(2)}$, and V through multi-scalar multiplication of length $O(\log_2 N)$ each on $\mathbb{G}_t$ during AggMEC, (3) running several times of Leopard_{PC}.Eval during Plonkish_{Eval}, and (4) checking the output of the AC for Plonkish is $[a]G \parallel [b]H$ through multi-scalar multiplication of length $2m$ on $\mathbb{G}_p$. Among them, we will focus on the third and fourth terms, whose computational cost depends on the choice of m, n for a fixed $N = mn$. Note that a single run of Leopard_{PC}.Eval for a polynomial of length D requires multi-scalar multiplication of $\sqrt{D}$ on both $\mathbb{G}_1$ and $\mathbb{G}_2$. With the batching technique, a single execution of Leopard_{PC}.Eval suffices during the entire AggMEC. Thus, if we denote the unit cost of each group operation in $\mathbb{G}_p, \mathbb{G}_1$, and $\mathbb{G}_2$ as c_p, c_1 , and c_2 , respectively, finding the optimal parameter is equivalent to solving the following optimization problem:

minimize:

$$C := (c_1 + c_2) \sqrt{12n \log_2 q \log_2 m + (8n - 4) \log_2 q} + c_p \cdot 2m$$

subject to: $N = mn$

Since dealing with C directly would be rather cumbersome because of the $\log_2 m$ and $(8n - 4) \log_2 q$ in the radical symbol, we instead consider minimizing

$$\widehat{C} = (c_1 + c_2) \sqrt{12n \log_2 q \log_2 N + 8n \log_2 q} + c_p \cdot 2m,$$

by replacing each term with $\log_2 N$ and $8n \log_2 q$, respectively. Now if we set $n = \frac{N}{m}$, $A = 2\sqrt{\log_2 q}$, and $B = 3N \log_2 N + 2N$, then we obtain

$$\widehat{C} = A \cdot \frac{c_1 + c_2}{2} \sqrt{\frac{B}{m}} + A \cdot \frac{c_1 + c_2}{2} \sqrt{\frac{B}{m}} + c_p \cdot 2m. \quad (14)$$

Hence, applying the AM-GM inequality in Eq. (14) yields the following inequality:

$$\begin{aligned} \widehat{C} &\geq 3 \cdot \sqrt[3]{2c_p \cdot \left(\frac{c_1 + c_2}{2}\right)^2 A^2 B} \\ &= 6c_p \sqrt[3]{3 \left(\frac{c_1 + c_2}{2c_p}\right)^2 \log_2 q \sqrt[3]{N \left(\log_2 N + \frac{2}{3}\right)}}, \end{aligned} \quad (15)$$

and equality holds when

$$m = \sqrt[3]{\frac{A^2 B}{4} \left(\frac{c_1 + c_2}{2c_p} \right)^2} \quad (16)$$

$$= \sqrt[3]{\left(\frac{c_1 + c_2}{2c_p} \right)^2 (3N \log_2 N + 2N) \log_2 q}.$$

We note that such parameter choice ensures that the computational complexity of the verifier is $O(\sqrt[3]{N \log_2 N})$, which is a slight improvement from $O(\sqrt[3]{N} \sqrt{\log_2 N})$ in Theorem V.1. In addition, to avoid the case when $m > N$, we select $m = N$ if the R.H.S. of Eq. (16) surpasses N .

For selecting m, n from the given N , we used the formula in Eq. (16) with measuring coefficients c_p, c_1 , and c_2 on our choice of elliptic curves. To this end, we first measured the ratio of unit group operations in our experimental environment, namely, $\mathbb{G}_p$ for Grumpkin and $\mathbb{G}_1, \mathbb{G}_2$ for BN254. We evaluate the time elapsed for 1 million group operations in each group. For $c_p = 1$, we obtain $c_1 = 1.0$ and $c_2 = 3.2$ with a 1-sigma error from 100 runs of the experiments. In addition, since points in the Grumpkin curve can be represented as a tuple of 254-bit integers, we set $\log_2 q = 256$, which is the closest power of two to 254. We report the choice of m, n , and D for $N = 2^{10}$ to $N = 2^{20}$ in Table 6.

TABLE 6. Selected parameters m, n and corresponding D for witnesses of length N . We set N around 2^{10} to 2^{20} . Each parameter is chosen by the formula in Eq. (16).

N	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}	2^{17}	2^{18}	2^{19}	2^{20}
m	2^8	2^9	2^9	2^9	2^{10}	2^{10}	2^{10}	2^{11}	2^{11}	2^{11}	2^{13}
n	2^2	2^3	2^3	2^4	2^4	2^5	2^6	2^6	2^7	2^8	2^8
D	2^{17}	2^{17}	2^{18}	2^{19}	2^{19}	2^{20}	2^{21}	2^{22}	2^{23}	2^{24}	2^{24}

In this section, we provide the optimization tricks to improve the prover and verifier cost of Cougar, which were omitted in the main text due to page limit.

B. BATCHED EXECUTION OF LEOPARD PCS

During AggMEC, the prover needs to execute Leopard PCS Leopard_{PC}.Eval on several polynomials and various evaluation points. In fact, by employing the techniques used in [10], [15], these processes can be done in a batched manner. To convince the verifier that the evaluations of several polynomials $f_1(X), \dots, f_k(X)$ at different evaluation points $u_1, \dots, u_k$ are $v_1, \dots, v_k$, respectively, it suffices to show the existence of polynomials $q_1(X), \dots, q_k(X)$ such that $f_i(X) - v_i = q_i(X)(X - u_i)$. Following the idea of [10], [15], the prover (1) constructs $\hat{q}_i(X) := \sum_{j=1}^k r_1^{i-1} q_j(X)$ for a random challenge r_1 received from the verifier, (2) commits $\hat{q}(X)$, (3) opens $f_1(X), \dots, f_k(X)$, and $\hat{q}(X)$ at another random point r_2 provided by the verifier, sending $w_1 := f_1(r_2), \dots, w_k := f_k(r_2)$ and $w_{k+1} := \hat{q}(r)$ to the verifier. Now, the verifier convinces the prover's claim if $w_{k+1} = \sum_{i=1}^k r_1^{i-1} \frac{w_i - v_i}{r_2 - u_i}$ holds.

We note that a single run of Leopard_{PC}.Eval suffices for the third step through a random linear combination on the commits of each polynomial. More precisely, for a random challenge r_3 received from the verifier, it suffices to ensure

that the evaluation of $\hat{f}(X) = \sum_{i=1}^k r_3^{i-1} f_i(X) + r_3^k \hat{q}(X)$ at r_2 is $\sum_{i=1}^k r_3^{i-1} w_i + r_3^k w_{k+1}$. Since Leopard_{PC} is a homomorphic commitment, the commitment of $\hat{f}$ can be computed from those of $f_1, \dots, f_k$ and $\hat{q}$, i.e., the verifier can be computed itself.

Thus, only a single execution of Leopard_{PC}.Eval suffices for AggMEC, along with a constant number of group operations on $\mathbb{G}_t$ for merging commitments.

C. MERGING MULTI-SCALAR MULTIPLICATIONS

The previous paragraph addressed the number of required group operations on $\mathbb{G}_p, \mathbb{G}_1$, and $\mathbb{G}_2$. We now move our focus to that on $\mathbb{G}_t$. Although the asymptotic required number of $\mathbb{G}_t$ during the whole protocol is $O(\log_2 N)$, the effect of them on the concrete efficiency of the verifier would become non-negligible because of the relatively expensive unit cost for the group operation. We can precisely count the input length of each multiscalar multiplication on $\mathbb{G}_t$ during AggMEC as follows:

- $3 \log_2 m + \log_2 n + 1$ for computing each $A^{(i)}, i \in \{1, 2\}$.
- $6 \log_2 m + 2 \log_2 n + 2$ for computing V .
- 5 for computing $P = V + [\tau]A^{(1)} + [\tau^2]A^{(2)} + [\tau^3]Q^{(1)} + [\tau^4]Q^{(2)}$.
- 36 for random linear combination during the batching technique.
- $2 \log_2 D$ for the final process of Leopard_{PC}.Eval.

Our strategy to address this is to merge multiscalar multiplications on $\mathbb{G}_t$ into a single but longer multiscalar multiplication. We note that the cost of multiscalar multiplication is sublinear to the length of the input vector [48].

To this end, we observed that with the batched evaluation technique, the verifier does not need to compute P earlier. Computing them before running the batched Leopard_{PC}.Eval at the end of AggMEC is sufficient. In addition, if we take a closer look at the batching process, we can figure out that the verifier computes the linear combination of a bulk of commitments before running the final process. Finally, at the end of Leopard_{PC}.Eval, the verifier needs to check the consistency of the final commitment by using challenges and other commitments communicated during the protocol (line 18 in Algorithm 6). We note that the former two processes can be delayed because they do not affect the process of the remaining protocol except for the last batched Leopard_{PC}.Eval. Hence, the verifier can postpone these multiscalar multiplications and merge them to the last part of Leopard_{PC}.Eval. Moreover, note that V and $[\tau]A^{(1)} + [\tau^2]A^{(2)}$ share the same base group points: $L_i^{(c)}, R_i^{(c)}$, and $P_i^{(c)}$ for $c \in \{1, 2\}$. Thus, it suffices for the verifier to compute a multiscalar multiplication of length $(2 \log_2 D + 6 \log_2 m + 2 \log_2 n + 40)$ at the last of Leopard_{PC}.Eval.

D. PRECOMPUTATION FOR EFFICIENT COMMITMENT

Our homomorphic commitment Com₂ requires $O(D)$ computation independent of the length of the vectors. To avoid the prover's large computation, we apply the precomputation

technique in [63]. That is, the committer computes commitment $(H_i = \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, H_i(X)))$, where $H_i(X) = \prod_{j \neq i} (\frac{1}{\zeta^j - \zeta^i}) \cdot \frac{X^D - 1}{X - \zeta^i}$ for all i . Then, the commitment to $L \in \mathbb{Z}_q^N$ can be computed by N linear computation of (H_i) . Hereafter, we consider the computation cost of Com_2 for N -length message vector as $O(N)$.

APPENDIX C PLONKISH ARITHMETIZATION FOR COUGAR

To check the elliptic curve relation at Eq. (4) and Eq. (5), we leverage Plonkish, which enables us to represent the circuit satisfiability of the given AC as a constraints system using *custom gates*. In this section, we provide our design of custom gates for elliptic curve relations, along with the resulting Plonkish proof system.

A. HIGH-LEVEL IDEA OF PLONKISH ARITHMETIZATION

Since Plonkish is a generalization of Plonk [30], we first provide a basic idea of it. For each gate in the circuit, Plonk constructs a constraint equation according to the gate type, *e.g.*, addition or multiplication, of the gate. Selecting the gate type is implemented by an auxiliary variable called the *selector*. Plonk manages each input/output of gates by constructing an execution trace, whose row represents the inputs, outputs, and auxiliary values that occurred in the corresponding gate execution step. By interpolating each column of the execution trace, along with the permutation constraints to connect one gate's output to the others' input, all the constraints can be represented as a single polynomial equation.

Plonkish follows the same blueprint as Plonk, generalizing it by employing custom gates other than addition or multiplication. Custom gates may require additional inputs, outputs, or auxiliary values, or need more computational depth for a single evaluation. Hence, the execution trace of Plonkish would have more columns than that of Plonk, and the degree of polynomials at the final polynomial equation representation of the constraint system relies on the (maximum) degree of the custom gates.

B. COUGAR-FRIENDLY CUSTOM GATES

We now present Plonkish-type constraint equations and custom gates for proving the elliptic curve relations. We will denote the degree of an operation as the depth of the arithmetic circuit with respect to multiplication gates.

a: Projective-Affine Mixed Operation Custom Gates

To ensure the elliptic curve relations Eq. (4) and (5), the prover should construct constraint systems for elliptic curve additions and doublings. As presented in halo2 [64], the custom gates for them on the affine representation would reduce the number of input gates. However, many implementations of elliptic curve groups use group operations on projective coordinates [51] to improve efficiency by avoiding modular inversions. In particular, they utilize *mixed* addition to further reduce the number of field operations needed, which is an

elliptic curve addition for two curve points of a projective representation and an affine representation, respectively, returning their sum in projective representation. Therefore, using custom gates for affine representation would be less efficient compared to the case without considering constructing an execution trace.

We thus aim to construct a custom gate for mixed elliptic curve operations presented in [51]. For a projective point $P = (X_1, Y_1, Z_1)$ and an affine point $Q = (X_2, Y_2)$ on the prime-order short Weierstrass elliptic curve $y^2 = x^3 + b^4$, the mixed addition $P + Q = (X_3, Y_3, Z_3)$ can be computed by

$$\begin{aligned} X_3 &= (X_1 Y_2 + X_2 Y_1)(Y_1 Y_2 - 3bZ_1) - 3b(Y_1 + Y_2 Z_1)(X_1 + X_2 Z_1), \\ Y_3 &= (Y_1 Y_2 + 3bZ_1)(Y_1 Y_2 - 3bZ_1) + 9bX_1 X_2 (X_1 + X_2 Z_1), \\ Z_3 &= (Y_1 + Y_2 Z_1)(Y_1 Y_2 + 3bZ_1) + 3X_1 X_2 (X_1 Y_2 + X_2 Y_1). \end{aligned}$$

In addition, the point doubling $[2]P = (X_3, Y_3, Z_3)$ can also be performed by

$$\begin{aligned} X_3 &= 2X_1 Y_1 (Y_1^2 - 9bZ_1^2), \\ Y_3 &= (Y_1^2 - 9bZ_1^2)(Y_1^2 + 3bZ_1^2) + 24bY_1^2 Z_1^2, \\ Z_3 &= 8Y_1^3 Z_1. \end{aligned}$$

Note that these operations are of degree 4, and [51, Algorithm 8, 9] provided efficient algorithms for computing them via field additions and multiplications. By using these operations, we can compute the scalar multiplication on the elliptic curve through the well-known double-and-add algorithm as presented in Algorithm 4. Since the doubling for the first iteration of the loop always results in the identity point, we omit it when configuring an execution trace for this operation.

Algorithm 4 Double-and-Add for Scalar Multiplication

Input: $P = (X, Y, 1)$ on $E : y^2 z = x^3 + axz^2 + bz^3$ defined over $\mathbb{Z}_p$ and $k \in \mathbb{Z}_q$.

- 1: Initialize $R \leftarrow (0, 1, 0)$
- 2: Compute $(k_1, \dots, k_{\lceil \log_2 q \rceil}) \in \{0, 1\}^{\lceil \log_2 q \rceil}$ such that $k = \sum_{i=1}^{\lceil \log_2 q \rceil} k_i 2^{i-1}$.
- 3: For i from $\lceil \log_2 q \rceil$ to 0 do:
- 4: Update $R \leftarrow [2]R + [k_i]P$.
- 5: **Return** R .

b: Consistency Check between Representations

We note that the second layer of our two-tier commitment scheme treats each elliptic curve group element as a tuple of field elements in an affine coordinate. Hence, the final result of the custom gate should be presented in affine point representation, and we need an additional constraint equation to confirm the equivalence of group elements in affine and projective representations. For two points $P = (X_1, Y_1, Z_1)$ and $Q = (X_2, Y_2)$, this can be done by following degree 2 operations.

$$X_1 = X_2 Z_1, \quad Y_1 = Y_2 Z_1$$

⁴Of course, [51] provided general results for curves of the form $y^2 = x^3 + ax + b$. We presented a special case for the sake of efficiency.

	S_1	S_2	C_1	C_2	C_3	C_4	C_5	
	1	0			0	1	0	Doubling
	1	1	$P^{(1)}$	$P^{(2)}$	0	1	0	Mixed Addition
	1	0			$P^{(1)}$	$P^{(2)}$	$P^{(3)}$	Doubling
	1	1	$P^{(1)}$	$P^{(2)}$	$[2]P^{(1)}$	$[2]P^{(2)}$	$[2]P^{(3)}$	Mixed Addition
	1	0			$[3]P^{(1)}$	$[3]P^{(2)}$	$[3]P^{(3)}$	Doubling
	0	1			$[6]P^{(1)}$	$[6]P^{(2)}$	$[6]P^{(3)}$	Identity Addition
	0	0	$[6]P^{(1)}$	$[6]P^{(2)}$	$[6]P^{(1)}$	$[6]P^{(2)}$	$[6]P^{(3)}$	Final Check

1

1

0

$6 = (110)_2$

FIGURE 4. Description of the execution trace for computing $[6]P$.

In addition, since at least one of the two input points should be represented as an affine coordinate for all operations, such a position would be utilized to hold values of L_i , R_i , and P_i for a consistency check during AggMEC. Therefore, all of these operations can be implemented at most degree 4 ACs, harmonizing well with our Cougar construction.

C. CONSTRUCTING EXECUTION TRACE

We now construct a Plonkish-style execution trace with the custom gates discussed above. Our execution trace consists of seven columns, where two columns are assigned for two selectors, S_1 and S_2 , and the remaining ones are for one projective point $P = (X_1, Y_1, Z_1)$ and one affine point $Q = (X_2, Y_2)$, respectively. We used two selectors for four types of operations, namely, addition (Add), addition by identity (AddById), doubling (Double), and equality check (EqCheck). For each operation, we assign the values of selectors as $(S_1, S_2) = (1, 1)$, $(0, 1)$, $(1, 0)$, and $(0, 0)$, respectively. Of course, it is more natural to employ four selectors corresponding to these four operations; we decided to use two selectors to reduce the number of columns. Moreover, because the outputs of mixed addition and doubling are points in projective coordinates, we assign the output of these operations to the next row of columns for storing projective points. This also saves the number of columns. To help understand, we provide an execution trace for checking $Q = [6]P$ in Fig. 4. In this figure, $[k]P^{(i)}$ means the i th coordinate of the point $[k]P$, and the prime symbol means the affine representation of the curve point. The points without the prime symbol are the projective representations. For the general case $Q = [k]P$ for $k \in \mathbb{Z}_q$, exactly $2\lceil \log q \rceil$ rows are sufficient for constructing the execution trace.

1) Compatibility with Plonk-Friendly Encoding.

With the above construction, the prover can construct the desired execution trace to ensure the elliptic curve relations. However, to make the execution trace compatible with the plonk-friendly encoding technique, the prover should carefully place each L , R , and P in the right position for each ζ^i .

To address this, we take a closer look at the equations in each relation, Eq. (4) and (5). First, in the k 'th round of the row reduction (Eq. (4)), the prover should construct an execution trace to ensure the relation

$$P_{k+1,i} = [x_k^{-1}]L_{k+1,i} + P_{k,i} + [x_k]R_{k+1,i}$$

for the challenge x_k and each i 'th coordinate of P_{k+1} , L_{k+1} , P_k , and R_{k+1} , respectively. To this end, the prover can compute this in order of computing (1) $\widetilde{L}_{k+1,i} := [x_k^{-1}]L_{k+1,i}$, (2) $R_{k+1,i} := [x_k]R_{k+1,i}$, (3) $P_{k+1,i} := \widetilde{L}_{k+1,i} + P_{k,i} + R_{k+1,i}$, and checking (4) $P_{k+1,i} = \widetilde{P}_{k+1,i}$. Here, the permutation argument is required for processing the third step because the outputs of the first step ($\widetilde{L}_{k+1,i}$) and the second step ($R_{k+1,i}$) should be referred to. According to our construction, each first and second step requires $2\lceil \log q \rceil$ rows, and each $L_{k+1,i}$ and $R_{k+1,i}$ appear at the first row of each position. Hence, by computing $\widetilde{L}_{k+1,i} + P_{k,i}$ first during the third step, the prover can coordinate each $L_{k+1,i}$, $R_{k+1,i}$, and $P_{k,i}$ with the period of $2\lceil \log q \rceil$. Of course, the remaining rows after the fourth step can be filled with an appropriate number of dummy operations, e.g., copying the check operations in the fourth step. Therefore, the execution trace for convincing the whole Eq. (4) can be constructed by successively iterating the above process over each coordinate and reduction stage.

The execution trace for the column reduction (Eq. (5)) can also be constructed in a similar manner, or even simpler. For the k 'th round of reduction, the prover needs to convince the following relation

$$P_{k+1,i} = P_{k,i}^{(q_1)} + [x_k]P_{k,i}^{(q_2)}, \quad P_{k+1,i+l} = P_{k,i}^{(q_3)} + [x_k^{-1}]P_{k,i}^{(q_4)},$$

for the challenge x_k and each coordinate of P_{k+1} , $P_k^{(q_1)}$, $P_k^{(q_2)}$, $P_k^{(q_3)}$, and $P_k^{(q_4)}$, respectively. Here, we assumed that the length of P_{k+1} is $2l$ and that for each $P_k^{(q_i)}$ is l . Similar to above, the prover can compute this by computing (1) $\widetilde{P}_{k+1,i} := [x_k]P_{k,i}^{(q_2)} + P_{k,i}^{(q_1)}$, and checking (2) $P_{k+1,i} = \widetilde{P}_{k+1,i}$. The same procedure suffices for computing $P_{k+1,i+l}$. In this case, the permutation argument is not required, and the construction of the execution trace can be done in the same way as above via dummy operations appropriately.

2) Reducing Dummy Rows

The excessive use of dummy operations results in more rows in the execution trace, thus increasing the computational cost for running Leopard_{PC}.Eval. To mitigate this, we present a more efficient construction of the execution trace by reducing dummy rows as much as possible.

In the column reduction process, each component of P_k does not need to be placed with the same period as that for the row reduction. Recall that the goal of Plonk-friendly encoding is to facilitate the check of the membership of each L_k , R_k , and P_k on the wire polynomial. Thus, we place each P_k in the column reduction with the same period as L_k , R_k , and P_k in the row reduction. This would not require modifying the previous AggMEC algorithm while saving $2dn$ rows.

In addition, we place each L_k , R_k , and P_k in the same order of their index to simplify the indexing. For the row reduction, we placed input vectors L_{k+1} , R_{k+1} , and P_k with a period of $2\lceil \log_2 q \rceil$. On the other hand, because the order of recorded components for column reduction does not match the input vector, we instead place the output P_{k+1} of the process with

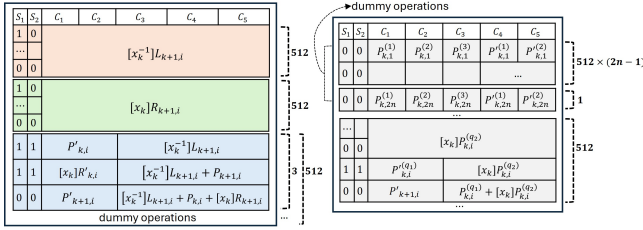


FIGURE 5. Final construction of the execution trace.

the same period. However, in this case, the output $P_{\mu+1}$ of the last row reduction, *i.e.*, the input of the first column reduction, is not recorded. We mitigate this by manually recording each component of $P_{\mu+1}$ into the execution trace with dummy rows containing these components. To match the period, we add an appropriate number of dummy rows except for the last component $P_{\mu+1,2n}$. This is because the next component $P_{\mu+2,1}$, the output of the first column reduction, should be separated from $P_{\mu+1,2n}$ with the period.

Note that the execution trace should contain at least $(6n \log_2 m + 4n - 2)d$ rows as it records each L_i , R_i , and P_i . That is, our optimization provides the optimal number of rows. We illustrated the final construction of the execution trace in Fig. 5 for a concrete case when $\log_2 q = 256$, *i.e.*, each $L_{k,i}$, $R_{k,i}$, and $P_{k,i}$ is placed with a period of 512.

D. FULL PLONKISH PROOF SYSTEM FOR COUGAR

We finally provide the full description of Plonkish proof system for the circuit satisfiability of the elliptic curve relations. The description is provided in Algorithm 5, and we will provide details in the following paragraphs.

a: Constraint Equations for Custom Gates

Let $\{w^{(i)}(X)\}$ be the polynomial corresponding to i^{th} column of the execution trace, including selector variables, for $i = 1, \dots, 5$. We will interpolate each column via a root of unity ξ . We denote $w^{(1)}, w^{(2)}$ as the elliptic curve point with affine representation, while $w^{(3)}, w^{(4)}, w^{(5)}$ are with affine representation. We also denote $\hat{s}_1(X)$ and $\hat{s}_2(X)$ as the polynomials that contain the selector values for each row. Note that selector bits (1, 1), (0, 1), (1, 0), (0, 0), corresponds to Add, AddByld, Double, and EqCheck operations, respectively. Let us denote corresponding constraint equations as $c_{\text{Add}}, c_{\text{AddByld}}, c_{\text{Double}},$ and c_{EqCheck} , respectively.

From this setting, we design the full constraint equation to ensure the relationship between the computation result and certain values in the same or next row of the execution trace. For example, for the Add custom gate, we need to ensure that the evaluation result is well placed in the next execution trace when the corresponding selector is activated. More precisely, if we denote $(v^{(3)}(X), v^{(4)}(X), v^{(5)}(X))$ as the output of $c_{\text{Add}}(w^{(1)}(X), \dots, w^{(5)}(X))$, then

$$\hat{s}_1(X)\hat{s}_2(X)v^{(j)}(X) = \hat{s}_1(X)\hat{s}_2(X)w^{(j)}(\xi X)$$

should hold for all ξ^i and indices $j \in \{3, 4, 5\}$. Note that we can represent it as a single polynomial equation via a

random linear combination with the challenge provided by the verifier, namely,

$$\hat{s}_1(X)\hat{s}_2(X) \left[\sum_{i=3}^5 \gamma^{i-3} \{v^{(i)}(X) - w^{(i)}(\xi X)\} \right] = 0.$$

From the above, we define the constraint polynomial g_{Add} as

$$g_{\text{Add}}(w^{(1)}(X), \dots, w^{(5)}(X); \gamma) = \sum_{i=3}^5 \gamma^{i-3} \{v^{(i)}(X) - w^{(i)}(\xi X)\}.$$

In addition, we can regard $s_{\text{Add}}(X) := \hat{s}_1(X)\hat{s}_2(X)$ as the selector polynomial for Add. The selector for other gates can be defined as

$$s_{\text{AddByld}}(X) = (1 - \hat{s}_1(X))\hat{s}_2(X)$$

$$s_{\text{Double}}(X) = \hat{s}_1(X)(1 - \hat{s}_2(X))$$

$$s_{\text{EqCheck}}(X) = (1 - \hat{s}_1(X))(1 - \hat{s}_2(X)).$$

We can design constraint polynomials for AddByld and Double, say g_{AddByld} and g_{Double} , respectively, in a similar manner by defining $(v^{(3)}(X), v^{(4)}(X), v^{(5)}(X))$ according to the gate. Here, note that EqCheck does not require referring to the next row of the execution trace. Instead, for a random challenge γ from the verifier, it satisfies

$$w^{(1)}(X) - w^{(3)}(X)w^{(5)}(X) + \gamma \cdot (w^{(2)}(X) - w^{(4)}(X)w^{(5)}(X)) = 0.$$

Hence, we define g_{EqCheck} like other constraint polynomials.

For simplicity, in Algorithm 5, we will identify Add, AddByld, Double, and EqCheck to 1, 2, 3, and 4, respectively. That is, we set $N_g = 4$ and $M = 5$.

b: Constraint Equations for Permutation

When executing the third step in computing Eq. (4), the prover should ensure that each elliptic curve point is correctly referenced from the previous execution trace. Hence, Plonkish employs an additional constraint equation based on *permutation*, *i.e.*, the permutation between the original and referenced variables does not change the evaluation result. A typical strategy is to check that the product of all the values is the same between the original and permuted ones. Likewise to the formulation in line 3, the prover computes the accumulation of the product for each index and convinces the verifier that (i) consecutive indices have a product relationship and (ii) the last index contains 1.

c: Merging Them All Together

To simultaneously convince the satisfiability of the constraint equations, the prover finally aggregates all the equations through random linear combination via the verifier's challenge, obtaining a polynomial $t(X)$. It should be ensured that $t(\xi^i) = 0$ for all the primitive roots. To show this, the prover shows the existence of a polynomial $q(X)$ such that $t(X) = q(X)(X^D - 1)$. This can be verified by evaluating $t(X)$ and $q(X)$ at a random point, say u_4 , and check that

Algorithm 5 Plonkish_{Eval}

- Plonkish_{Eval}(ck_{PC}, {s_i(X), g_i(X₁, ..., X_M)_{i=1}^{N_g}, {r_i(X)}_{i=1}^M; {w⁽ⁱ⁾(X)}_{i=1}^M)
- Precompute:** C_{ID_i} = Comp_{PC}(ck_{PC}, ID_i(X)), C_{r_i} = Comp_{PC}(ck_{PC}, r_i(X)), i ∈ [M]
- 1: $\mathcal{P}$ sends $W^{(i)} = \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, w^{(i)}(X))$ to $\mathcal{V}$
 - 2: $\mathcal{V}$ chooses $u_1, u_2 \xleftarrow{\$} \mathbb{Z}_q$ and sends it to $\mathcal{P}$
 - 3: $\mathcal{P}$ sends $Z = \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, z(X))$ to $\mathcal{V}$ where
$$z(X) = H_0(X) + \sum_{j=0}^{D-2} \left(H_{j+1}(X) \prod_{h=0}^j \prod_{i=1}^M \frac{w^{(i)}(\xi^h) + u_1 \text{ID}_i(\xi^h) + u_2}{w^{(i)}(\xi^h) + u_1 r_i(\xi^h) + u_2} \right), \quad H_j(X) = \prod_{i \neq j, i \in [D]} \frac{X - \xi^i}{\xi^j - \xi^i} \text{ for all } j \in [D].$$
 - 4: $\mathcal{V}$ chooses $u_3 \xleftarrow{\$} \mathbb{Z}_q$ and $\eta_i \xleftarrow{\$} \mathbb{Z}_q$ for $i \in [N_g]$, and sends it to $\mathcal{P}$.
 - 5: $\mathcal{P}$ sends $T = \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, t(X))$, $Q = \text{Comp}_{\text{PC}}(\text{ck}_{\text{PC}}, q(X))$ to $\mathcal{V}$ where
$$t(X) = \sum_{i=1}^{N_g} s_i(X) g_i(w^{(0)}(X), \dots, w^{(M-1)}(X); \eta_i) + u_3 \cdot z(X) \prod_{i=1}^M (w^{(i)}(X) + u_1 \text{ID}_i(X) + u_2) \\ - u_3 \cdot z(\xi X) \prod_{i=1}^M (w^{(i)}(X) + u_1 r_i(X) + u_2) + u_3^2 \cdot (z(X) - 1) H_0(X)$$

$$q(X) = \frac{t(X)}{z_H(X)}, \text{ where } z_H(X) = \prod_{i=0}^{D-1} (X - \xi^i).$$
 - 6: $\mathcal{V}$ chooses $u_4 \xleftarrow{\$} \mathbb{Z}_q$ and sends it to $\mathcal{P}$.
 - 7: $\mathcal{P}$ sends $\{\alpha_i = w^{(i)}(u_4)\}_{i=1}^M, \beta = z(u_4), \gamma = z(\xi u_4),$
 $\{\phi_i = \text{ID}_i(u_4)\}_{i=1}^M$, and $\{\psi_i = r_i(u_4)\}_{i=1}^M$ to $\mathcal{V}$.
 - 8: $\mathcal{V}$ evaluates ρ_1 and $\rho_2 = \rho_1 / z_H(u_4)$ using the values received from $\mathcal{P}$

$$\rho_1 = \sum_{i=1}^{N_g} s_i(u_4) \cdot g_i(\alpha_1, \dots, \alpha_M; \eta_i) + u_3 \cdot \beta \prod_{i=1}^M (\alpha_i + u_1 \cdot \phi_i + u_2) \\ - u_3 \cdot \gamma \prod_{i=1}^M (\alpha_i + u_1 \cdot \psi_i + u_2) + u_3^2 \cdot (\beta - 1) H_0(u_4)$$
 - 9: $\mathcal{P}$ and $\mathcal{V}$ set run Eval(ck_{PC}, T, u₄, ρ₁; t(X)), Eval(ck_{PC}, W⁽ⁱ⁾, u₄, α_i; w⁽ⁱ⁾(X))_{i∈[M]},
Eval(ck_{PC}, Q, u₄, ρ₂; q(X)), Eval(ck_{PC}, Z, u₄, β; z(X)), Eval(ck_{PC}, Z, ξu₄, γ; z(X)),
Eval(ck_{PC}, C_{ID_i}, u₄, φ_i; ID_i(X))_{i∈[M]}, and Eval(ck_{PC}, C_{r_i}, u₄, ψ_i; r_i(X))_{i∈[M]}.

$t(u_4) = q(u_4)(u_4^D - 1)$, along with the opening proofs to ensure the validity of evaluation values.

Throughout this process, the prover also needs to convince the well-formedness of $t(X)$. Note that the prover has already committed the polynomials required for reconstructing $t(X)$, including columns of the execution trace, and original and permuted indices. Hence, the prover can open all these components at the same random point as above, and the verifier then checks whether the computed $t(u_4)$ coincides with the one computed from its components.

d: Handling Longer Polynomials

Committing $t(X)$ and $q(X)$ in line 5 would require a longer commitment key than D , namely, $\deg(t(X)) \cdot D$. Since the degree of all polynomials appearing in Cougar is at most D except for them, directly increasing the length of ck_{PC} incurs inefficiency for both the prover and verifier. As did in halo2 library [64], we address this issue by parsing $t(X)$ and $q(X)$ into polynomials of degree at most D and commit

them separately. Later, when opening these polynomials at u_4 , the prover needs to send additional evaluation points for each segment, along with opening proofs for each of them. Indeed, this modification carries additional computation/communications, but we can leverage the batching technique presented in Section B to aggregate the opening proofs and their verification once.

APPENDIX D LEOPARD PCS

In this section, we provide details about the Leopard_{PC}, which is a key ingredient to instantiate Cougar. Remark that [41, Section E.1.] provided a basic idea for constructing this; we present the full description for the sake of completeness.

Before the description of Leopard_{PC}, we additionally define the outer product notation as follows:

Outer-Pairing Product. We define an outer pairing product, which is a way to generate a target group matrix from source

group vectors. For $\mathbf{G} \in \mathbb{G}_1^m$ and $\mathbf{H} \in \mathbb{G}_2^n$, we denote

$$\mathbf{G} \otimes \mathbf{H} = \begin{bmatrix} e(\mathbf{G}_1, \mathbf{H}_1) & \cdots & e(\mathbf{G}_1, \mathbf{H}_n) \\ \vdots & \ddots & \vdots \\ e(\mathbf{G}_m, \mathbf{H}_1) & \cdots & e(\mathbf{G}_m, \mathbf{H}_n) \end{bmatrix} \in \mathbb{G}_t^{m \times n}$$

Leopard_{PC} is a natural tweak of Protocol3 [41] as a PCS. The construction idea is the same as the PCS introduced by [15], which was built upon BulletProofs. More precisely, we can construct PCS from IPA by regarding the point evaluation of the polynomial as an inner product between the coefficient vector and the vector comprised of the powers of the evaluation point. The asymptotic communication and computation complexities of Eval from this approach are the same as those of the underlying IPA. Note that Protocol3 features square root verifier cost and logarithmic communication cost; hence so does Leopard_{PC}.Eval.

Following the above approach, we provide the full description of Leopard_{PC} as follows: Let $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t)$ be a bilinear group, where $\mathbb{G}_1 = E(\mathbb{Z}_p)$. For a polynomial $a(X) \in \mathbb{Z}_p^{<mn}[X]$ and positive integers $m, n \in \mathbb{N}$, we will denote its coefficient vector as $\mathbf{a} \in \mathbb{Z}_p^{mn}$, namely, $\mathbf{a} = (a_0, \dots, a_{mn-1})$ such that $a(X) = \sum_{i=0}^{mn-1} a_i X^i$. Leopard_{PC} = (Gen, Com, Eval) over a message space $\mathbb{Z}_p^{<mn}[X]$ and a commitment space $\mathbb{G}_t$ is defined as follows⁵:

- Gen(1^λ) $\rightarrow$ ck_{PC} $\in \mathbb{G}_1^m \times \mathbb{G}_2^n$.
- Com(ck_{PC} = $(\mathbf{G}, \mathbf{H})$, $a(X)$) $\rightarrow P := [\mathbf{a}](\mathbf{G} \otimes \mathbf{H}) \in \mathbb{G}_t$.

In addition, Eval = $(\mathcal{K}, \mathcal{P}, \mathcal{V})$ is an interactive argument system for the following relation:

$$\mathcal{R}_{\text{Leopard}_{\text{PC}}.\text{Eval}} = \left\{ \left(\begin{array}{l} \text{ck}_{\text{PC}} = (\mathbf{G}, \mathbf{H}) \in \mathbb{G}_1^m \times \mathbb{G}_2^n, \\ C \in \mathbb{G}_t, z, y \in \mathbb{Z}_p, d \in [mn]; \\ a(X) \in \mathbb{Z}_p^{<mn}[X] \\ C = [\mathbf{a}](\mathbf{G} \otimes \mathbf{H}) \wedge y = a(z) \end{array} \right) : \right\}$$

A typical strategy to cope with the above relation is to modify the above relation into that for IPA: For $\mathbf{z} = (1, z, \dots, z^{mn-1})$, we can rewrite $a(z) = \langle \mathbf{a}, \mathbf{z} \rangle$. For this reason, the construction of Eval is almost identical to Protocol3 except for some modifications regarding the fact that $\mathbf{z}$ is also known to the verifier. The precise description of Leopard_{PC}.Eval is given in Algorithm 6. Here, bit(k) refers to the bit decomposition of a number k .

We now show that Leopard_{PC} is indeed the PCS, *i.e.*, satisfying the conditions in Definition II.4, under the DL assumption. In fact, $\mathbf{G} \otimes \mathbf{H}$ in the above relation can be seen as the commitment key of the Pedersen vector commitment defined over the group $\mathbb{G}_t$, along with a certain structure. Since the binding property of the Pedersen vector commitment depends on the DLR assumption, one can expect that the same holds for Leopard_{PC} under a structure-aware version of the DLR assumption.

As shown by [41, Theorem 5], if the DL assumption on both $\mathbb{G}_1$ and $\mathbb{G}_2$ hold, then the GDLR assumption also holds.

⁵We will not consider hiding property because zero-knowledge property is unnecessary in our context.

Algorithm 6 Leopard_{PC}.Eval

Leopard_{PC}.Eval(ck_{PC} = $(\mathbf{G}, \mathbf{H}) \in \mathbb{G}_1^m \times \mathbb{G}_2^n, P \in \mathbb{G}_t, z, y \in \mathbb{Z}_p; \mathbf{a} \in \mathbb{Z}_p^{mn}$)
 where $m = 2^\mu$ and $n = 2^\nu$

- 1: $\mathcal{V}$ picks $U \xleftarrow{\$} \mathbb{G}_t$ and sends it to $\mathcal{P}$
- 2: $\mathcal{P}$ and $\mathcal{V}$ set $P_0 = P + [y]U$, $\mathbf{G}_0 = \mathbf{G}$, $\mathbf{H}_0 = \mathbf{H}$.
 Additionally, $\mathcal{P}$ set $\mathbf{a}_0 = \mathbf{a}$ and $\mathbf{z}_0 = [z^{m(i-1)+(j-1)}] \in \mathbb{Z}_p^{m \times n}$
- 3: **for** $i = 0, \dots, \mu - 1$ **do**
- 4: $\mathcal{P}$ parses $\mathbf{a}_i, \mathbf{z}_i$, and $\mathbf{G}_i$ to
 $\mathbf{a}_i = [\mathbf{a}_{i,L} \parallel \mathbf{a}_{i,R}]$, $\mathbf{z}_i = [\mathbf{z}_{i,L} \parallel \mathbf{z}_{i,R}]$, $\mathbf{G}_i = \mathbf{G}_{i,L} \parallel \mathbf{G}_{i,R}$
- 5: $\mathcal{P}$ computes:
 $L_i = [\mathbf{a}_{i,L}](\mathbf{G}_{i,R} \otimes \mathbf{H}) + [\langle \mathbf{a}_{i,L}, \mathbf{z}_{i,R} \rangle]U \in \mathbb{G}_t$
 $R_i = [\mathbf{a}_{i,R}](\mathbf{G}_{i,L} \otimes \mathbf{H}) + [\langle \mathbf{a}_{i,R}, \mathbf{z}_{i,L} \rangle]U \in \mathbb{G}_t$
- 6: $\mathcal{P}$ sends L_i, R_i to $\mathcal{V}$
- 7: $\mathcal{V}$ chooses $r_i \xleftarrow{\$} \mathbb{Z}_p^*$ and sends it to $\mathcal{P}$
- 8: $\mathcal{P}$ computes:
 $\mathbf{a}_{i+1} = \mathbf{a}_{i,L} + r_i^{-1} \mathbf{a}_{i,R}$, $\mathbf{z}_{i+1} = \mathbf{z}_{i,L} + r_i \mathbf{z}_{i,R} \in \mathbb{Z}_p^{m/2^{i+1} \times n}$
 $\mathbf{G}_{i+1} = \mathbf{G}_{i,L} + [r_i] \mathbf{G}_{i,R} \in \mathbb{G}_1^{m/2^{i+1}}$
 $P_{i+1} = [r_i]L_i + P_i + [r_i^{-1}]R_i \in \mathbb{G}_t$
- 9: **end for**
- 10: **for** $j = 0, \dots, \nu - 1$ **do**
- 11: $\mathcal{P}$ sets $i = j + \mu$ and then parses $\mathbf{a}_i, \mathbf{z}_i$, and $\mathbf{H}_j$ to
 $\mathbf{a}_i = \mathbf{a}_{i,L} \parallel \mathbf{a}_{i,R}$, $\mathbf{z}_i = \mathbf{z}_{i,L} \parallel \mathbf{z}_{i,R}$, $\mathbf{H}_j = \mathbf{H}_{j,L} \parallel \mathbf{H}_{j,R}$
- 12: $\mathcal{P}$ computes:
 $L_i = [\mathbf{a}_{i,L}](\mathbf{G}_\mu \otimes \mathbf{H}_{j,R}) + [\langle \mathbf{a}_{i,L}, \mathbf{z}_{i,R} \rangle]U \in \mathbb{G}_t$
 $R_i = [\mathbf{a}_{i,R}](\mathbf{G}_\mu \otimes \mathbf{H}_{j,L}) + [\langle \mathbf{a}_{i,R}, \mathbf{z}_{i,L} \rangle]U \in \mathbb{G}_t$
- 13: $\mathcal{V}$ chooses $r_i \xleftarrow{\$} \mathbb{Z}_p^*$ and sends it to $\mathcal{P}$
- 14: $\mathcal{P}$ computes:
 $\mathbf{a}_{i+1} = \mathbf{a}_{i,L} + s_j^{-1} \mathbf{a}_{i,R}$, $\mathbf{z}_{i+1} = \mathbf{z}_{i,L} + s_j \mathbf{z}_{i,R} \in \mathbb{Z}_p^{n/2^{j-1}}$
 $\mathbf{H}_{j+1} = \mathbf{H}_{j,L} + [s_j] \mathbf{H}_{j,R} \in \mathbb{G}_2^{n/2^{j+1}}$
 $P_{i+1} = [s_i]L_i + \cdot P_i + [s_i^{-1}]R_i \in \mathbb{G}_t$
- 15: **end for**
- 16: $\mathcal{P}$ sends $a = a_{\mu+\nu} \in \mathbb{Z}_p$ to $\mathcal{V}$
- 17: $\mathcal{V}$ computes:
 $\mathbf{r}[k+1] = \langle \text{bit}(k), (r_0, \dots, r_{\mu+\nu-1}) \rangle$ for $k = 0, \dots, m + n - 1$
 Parse $\mathbf{r}$ to $\mathbf{r}_{\text{row}} \parallel \mathbf{r}_{\text{col}}$ where $\mathbf{r}_{\text{row}} \in \mathbb{Z}_p^m$ and $\mathbf{r}_{\text{col}} \in \mathbb{Z}_p^n$
 $\mathbf{G} = \langle \mathbf{r}_{\text{row}}, \mathbf{G}_0 \rangle$, $\mathbf{H} = \langle \mathbf{r}_{\text{col}}, \mathbf{H}_0 \rangle$, $\mathbf{z} = \mathbf{r}_{\text{row}} \mathbf{z}_0 \mathbf{r}_{\text{col}}$
- 18: $\mathcal{V}$ checks:
 $P_0 + \sum_{i \in [\mu+\nu]} ([r_i]L_i + [r_i^{-1}]R_i) = e([az]G, H)$

In addition, by assuming the GDLR assumption, the binding property of Leopard_{PC} holds immediately.

Now it remains to check that Leopard_{PC}.Eval is an AoK for the relation $\mathcal{R}_{\text{Leopard}_{\text{PC}}.\text{Eval}}$. As we mentioned, this relation can be understood as a special case of that for Protocol3, and Algorithm 6 is almost identical to Protocol3. We note that Protocol3 satisfies perfect completeness and computational witness-extended emulation under the GDLR assumption [40]. We made the same modifications as [15]

for constructing $\text{Leopard}_{\text{PC}}.\text{Eval}$, without considering zero-knowledge. That is, the proof strategies for computational witness-extended emulation of ours and theirs are identical, except for replacing the DLR assumption with the GDLR assumption. We refer to [40] and [15] for more detailed information.

To sum up, $\text{Leopard}_{\text{PC}}$ satisfies all conditions in Definition II.4 under the DL assumption on $\mathbb{G}_1$ and $\mathbb{G}_2$. In addition, it does not require the trusted setup and features squared root verification cost and logarithmic communication cost with respect to the length of the witness. Therefore, it is a desirable PCS for instantiating Cougar.

APPENDIX E MORE CONCRETE EFFICIENCY ANALYSIS

We provide a concrete efficiency analysis of the Cougar by incorporating all the optimization steps in Section B. We denote $\mathbb{G}^{\text{mul}}$ and $\mathbb{G}^{\text{exp}}$ as the cost for multiplication and exponentiation over the group, respectively. Note that computing $\mathbb{G}^{\text{exp}}$ requires $O(\log_2 |\mathbb{G}|)$ operations of $\mathbb{G}^{\text{mul}}$. We also denote P as the cost for a pairing operation. For simplicity, we calculate the cost of multiscalar multiplication of input size N as $N \cdot \mathbb{G}^{\text{exp}}$ over the group $\mathbb{G}$, though there is an efficient algorithm with sublinear complexity [48]. Similarly, we evaluate the cost for multi-pairing of length N as $N \cdot P$.

Likewise the analyses in Section V, we conduct cost analysis for each subroutine of Cougar: Protocol.Row , Protocol.Col , and AggMEC . In addition, recall that we split $N = mn$ and run $\mu := \log_2 m$ and $\nu := \log_2 n$ steps for Protocol.Row and Protocol.Col , respectively.

A. COSTS FOR SUBROUTINES

Recall that we exploit Plonkish and $\text{Leopard}_{\text{PCS}}$ as subroutines during the proof generation and verification. For this reason, before providing the full cost analysis of Cougar, we first analyze the concrete efficiency of each subroutine.

1) $\text{Leopard}_{\text{PCS}}.\text{Com}$

By definition, $\text{Leopard}_{\text{PCS}}.\text{Com}$ is defined as the multiscalar multiplication on the target group $\mathbb{G}_t$ on inputs $\mathbf{a} \in \mathbb{Z}_q^{n_1 \times n_2}$ and $\text{ck} = (\mathbf{G}, \mathbf{H}) \in \mathbb{G}_1^{n_1} \times \mathbb{G}_2^{n_2}$ for some $n = n_1 n_2$. As observed in Kim et al. [40], by leveraging the structure of the outer pairing product $\mathbf{G} \otimes \mathbf{H}$, one can compute it via multiscalar multiplications in $\mathbb{G}_1$ and the multipairing. For simplicity, let us denote $\mathbf{a}[i][j]$ as the entry in the i th row and j th column, and $\mathbb{G}[i]$ as the i th component of $\mathbb{G}$. In addition, we denote $\mathbf{a}[*][j]$ as the j th column vector of $\mathbf{a}$. Then we can observe that

$$\begin{aligned} [\mathbf{a}](\mathbf{G} \otimes \mathbf{H}) &= \prod_{i=1}^{n_1} \prod_{j=1}^{n_2} e(\mathbf{G}[i], \mathbf{H}[j])^{\mathbf{a}[i][j]} \\ &= \prod_{j=1}^{n_2} e \left(\prod_{i=1}^{n_1} \mathbf{G}[i]^{\mathbf{a}[i][j]}, \mathbf{H}[j] \right) \\ &= \prod_{j=1}^{n_2} e([\mathbf{a}[*][j]] \mathbf{G}, \mathbf{H}[j]). \end{aligned}$$

Hence, if we denote $\text{MP}(N)$ as the cost for multipairing of length N , we finally obtain the concrete cost for computing $[\mathbf{a}](\mathbf{G} \otimes \mathbf{H})$ as $n_2 \cdot n_1 \cdot \mathbb{G}_1^{\text{exp}} + \text{MP}(n_2)$. For the special case where $n_1 \approx n_2 \approx \sqrt{n}$, we will denote the cost for a single invocation of $\text{Leopard}_{\text{PCS}}.\text{Com}$ of input length N as

$$P_{\text{LCom}}(N) := N \cdot \mathbb{G}_1^{\text{exp}} + \sqrt{N} \cdot P.$$

2) Plonkish

According to Algorithm 5, Plonkish utilizes (Com, Eval) of $\text{Leopard}_{\text{PCS}}$ as subroutines. More precisely, we can summarize the required computations of both parties for each step as follows:

- Prover:
 - Compute C_{ID_i} , C_{r_i} , and $W^{(i)}$ for $i \in [M]$.
 - Compute $z(X)$, $t(X)$, $q(X)$ and their commitments.
 - Evaluation of $w^{(i)}(X)$, $z(X)$, $z(X)$, $\text{ID}_i(X)$, and $r_i(X)$ at designated points.
- Verifier: Compute ρ_1 and ρ_2 .

After then, they jointly runs Eval several times for polynomials $t(X)$, $q(X)$, $z(X)$, $q(X)$ and $w^{(i)}(X)$, $\text{ID}_i(X)$, $r_i(X)$ for $i \in [M]$. These Evals can be aggregated via random linear combination.

Here, note that the degree of $t(X)$ and $q(X)$ could be larger than D because each constraint polynomial g_i for $i \in [N_g]$ could have a higher degree than 1, say d_i . Hence, we apply the same trick as halo2 by chunking $t(X)$ and $q(X)$ into several parts and commit each of them separately, namely, write

$$t(X) = \sum_{i=1}^{d_t} X^{(i-1) \cdot D} t_i(X) \quad q(X) = \sum_{i=1}^{d_q-1} X^{(i-1) \cdot D} q_i(X),$$

and commit each $\{t_i(X)\}_{i=1}^{d_t}$ and $\{q_i(X)\}_{i=1}^{d_q-1}$. Thus, the prover should compute $3M + 2d_t$ Coms and both parties should run $3M + 2d_t + 1$ Evals in total.

Now we consider the case when merging those invocations of Eval at once. To this end, for the witness polynomials $f_1(X), \dots, f_k(X)$ and the evaluation points $v_1, \dots, v_k$, the prover first needs to commit $\hat{q}(X) := \sum_{i=1}^k r_1^{i-1} \frac{f_i(X) - f_i(v_i)}{X - v_i}$, where r_1 is the random challenge provided by the verifier. Later then, the prover opens $f_1(X), \dots, f_k(X)$ and $\hat{q}(X)$ at the same point r_2 provided by the verifier, which can be aggregated as opening $\hat{f}(X) := \sum_{i=1}^k r_3^{i-1} f_i(X) + r_3^k \hat{q}(X)$ for the other challenge r_3 . Recall that the commitment of $\hat{f}(X)$ can be homomorphically computed by the verifier. Throughout this procedure, the only additional process made by the prover is to commit $\hat{q}(X)$, and the verifier needs to compute the commitment of $\hat{f}(X)$ by computing a multiscalar multiplication over $\mathbb{G}_t$.

To sum up, the prover's computation required for the overall process is $(3M + 2d_t + 1) \cdot P_{\text{LCom}}(D) + 1 \cdot P_{\text{LEval}}(D)$. During aggregation, the verifier's computational cost involves computing a random linear combination of commitments for each $f_1(X), \dots, f_k(X)$, $\hat{q}(X)$ via a random challenge. In our context, since $(3M + 2d_t + 1)$ Evals are aggregated at once, the verifier's cost becomes $(3M + 2d_t + 2) \cdot \mathbb{G}_t^{\text{exp}}$.

B. PROVER COST

1) Protocol.Row

The protocol consists of μ steps in total, and for i^{th} step, $\hat{m} = \frac{m}{2^i}$. The prover's computation consists of the following three subroutines:

- 1) Computing $\mathbf{L}$, $\mathbf{R}$, c_L , and c_R from $\mathbf{a}$, $\mathbf{b}$, $\mathbf{G}$, and $\mathbf{H}$.
- 2) Computing $\mathbf{L}$, $\mathbf{R}$ from $\mathbf{L}$ and $\mathbf{R}$.
- 3) Updating $\mathbf{P}$, c , $\mathbf{a}$, $\mathbf{b}$, $\mathbf{G}$, $\mathbf{H}$ via a received challenge.

We analyze the computational cost for each step as follows.

First, when computing $\mathbf{L} = [\mathbf{a}_L] \mathbf{G}_R \| [\mathbf{b}_R] \mathbf{H}_L$, each $[\mathbf{a}_L] \mathbf{G}_R$ consists of n multiscalar multiplications of length $\frac{m}{2^i}$ over $\mathbb{G}_p$. Hence, the cost for computing $\mathbf{L}$ and $\mathbf{R}$ is $\frac{m}{2^{i-2}} \cdot \mathbb{G}_p^{\text{exp}}$ in total. In addition, computing c_L and c_R consists of $\frac{mn}{2^i}$ operations over $\mathbb{Z}_p$ each. To sum up, the cost for the first step is $\frac{m}{2^{i-2}} \cdot \mathbb{G}_p^{\text{exp}} + \frac{mn}{2^{i-1}} \cdot \mathbb{Z}_p$.

Second, computing $\mathbf{L}$ and $\mathbf{R}$ involves invoking `LeopardPCS.Com` on inputs of size $2n$. Hence, the required computation cost becomes $2 \cdot P_{\text{LCom}}(2n)$.

Finally, the update of $\mathbf{P}$ requires computing $\hat{\mathbf{P}}$ first and invoke `LeopardPCS.Com` once, which requires $4n \cdot \mathbb{G}_p^{\text{mul}} + 4n \cdot \mathbb{G}_p^{\text{exp}} + P_{\text{LCom}}(2n)$. Updating $\mathbf{a}$, $\mathbf{b}$, c requires $(\frac{mn}{2^{i-1}} + 2) \cdot \mathbb{Z}_p$. Updating $\mathbf{G}$, $\mathbf{H}$ requires $\frac{mn}{2^{i-1}} \cdot \mathbb{G}_p^{\text{mul}} + \frac{m}{2^{i-1}} \cdot \mathbb{G}_p^{\text{exp}}$.

Therefore, by running i from 1 to $\mu = \log_2 m$, we can summarize the prover's concrete computational cost for Protocol.Row as

$$\sum_{i=1}^{\log_2 m} \left[3 \cdot P_{\text{LCom}}(2n) + \left(\frac{m}{2^i} \cdot 4n + 2 \right) \cdot \mathbb{Z}_p + \frac{m}{2^{i-2}} \cdot \mathbb{G}_p^{\text{exp}} + (4n + \frac{m}{2^{i-1}}) \cdot \mathbb{G}_p^{\text{mul}} + (4n + \frac{m}{2^{i-1}}) \cdot \mathbb{G}_p^{\text{exp}} \right]$$

$$= \frac{3 \log_2 m \cdot P_{\text{LCom}}(2n) + (4mn + 2 \log_2 m) \cdot \mathbb{Z}_p + (4n \log_2 m + m) \cdot \mathbb{G}_p^{\text{mul}} + (4n \log_2 m + 6m - 6) \cdot \mathbb{G}_p^{\text{exp}}}{}$$

2) Protocol.Col

The protocol consists of ν steps in total, and for j^{th} step, $\hat{n} = \frac{n}{2^j}$. Likewise Protocol.Row, the prover's computation is comprised of

- Computing c_L and c_R from $\mathbf{a}$ and $\mathbf{b}$.
- Updating $\hat{\mathbf{P}}$, $\mathbf{a}$, $\mathbf{b}$ via a received challenge.

The first process requires $\frac{n}{2^{j-1}}$ computations over $\mathbb{Z}_p$. The update of $\hat{\mathbf{P}}$ involves computing $\hat{\mathbf{P}}$ and running `LeopardPCS.Com`. Each operation requires $\frac{n}{2^{j-1}} \cdot \mathbb{G}_p^{\text{mul}} + \frac{n}{2^{j-1}} \cdot \mathbb{G}_p^{\text{exp}}$ and $1 \cdot P_{\text{LCom}}(\frac{n}{2^j})$, respectively. Updating $\mathbf{a}$ and $\mathbf{b}$ requires $\frac{n}{2^{j-1}}$ operations in $\mathbb{Z}_p$ total.

Thus, by running j from 1 to $\nu = \log_2 n$, the prover's total computation for Protocol.Col becomes

$$\sum_{j=1}^{\log_2 n} \left[\frac{n}{2^{j-1}} \cdot \mathbb{Z}_p + \frac{n}{2^{j-2}} \cdot \mathbb{G}_p^{\text{exp}} + \frac{n}{2^{j-1}} \cdot \mathbb{G}_p^{\text{mul}} + P_{\text{LCom}}(\frac{n}{2^j}) \right]$$

$$= \frac{(2n - 2) \cdot \mathbb{Z}_p + (4n - 4) \cdot \mathbb{G}_p^{\text{exp}} + (2n - 2) \cdot \mathbb{G}_p^{\text{mul}} + \sum_{j=1}^{\log_2 n} P_{\text{LCom}}(\frac{n}{2^{j-1}})}{}$$

3) AggMEC

In AggMEC, the prover's computation consists of the following three steps:

- 1) Prepare aggregated commitments $A^{(i)}$ and its opening polynomials. (line 1–2)
- 2) Prepare auxiliary values for proving the consistency between the commitment and the wire polynomial in Plonkish. (line 3–11)
- 3) Run `PlonkishEval` and `LeopardPC.Eval`. (line 12)

The first step requires aggregating all the commitments L_k, R_k, P_k that were sent to the verifier during Protocol.Row and Protocol.Col. Since the prover sends L_k, R_k for $k = 1, \dots, \log_2 n$, whereas P_l for $l = 1, \dots, \log_2 n + \log_2 m$, computing $A^{(i)}$ requires $(2 \log_2 n + \log_2 m) \cdot \mathbb{G}_t$ for $i = 1, 2$. In addition, preparing polynomials $a^i(X)$ requires invoking an NTT of length D , where D is the number of rows in the execution trace for Plonkish.

The second step involves (i) computing $Q^{(i)}$ from $W^{(i)}$, (ii) computing aggregated commitment V from L_k, R_k, P_k with the verifier's random challenge ρ , and (iii) computing the final aggregated commitment P from $V, A^{(i)}$, and $Q^{(i)}$. The first sub-step requires invoking `LeopardPC.Eval` of length D , the second sub-step requires computing a MSM of size $(4 \log_2 n + 2 \log_2 m)$ and the third sub-step requires computing an MSM of size 4. To sum up, the prover's required computation is $P_{\text{LCom}}(D) + (4 \log_2 n + 2 \log_2 m + 4) \cdot \mathbb{G}_t^{\text{exp}}$.

The third step involves two runs of `LeopardPC.Eval` of size D and one `PlonkishEval`. As we described earlier, `PlonkishEval` involves $(3M + 2d_t + 1)$ runs of `LeopardPC.Eval` of size D .

Hence, we can conclude that the total number of required computations is

$$(8 \log_2 n + 4 \log_2 m + 4) \cdot \mathbb{G}_t^{\text{exp}} + D \log_2 D \cdot \mathbb{Z}_p + (3M + 2d_t + 3) \cdot P_{\text{LCom}}(D)$$

Note that we have provided the concrete value of D at Eq. 13.

C. VERIFIER COST

Throughout the protocol, the verifier needs to check all the relations that appear in Protocol.Row and Protocol.Col, along with several `LeopardPC.Eval` and `PlonkishEval` at the end of the protocol. First, for Protocol.Row, the verifier needs to update $\mathbf{G}$ and $\mathbf{H}$ according to the challenges, which requires $\frac{m}{2^{i-1}} \cdot \mathbb{G}_p^{\text{exp}} + \frac{m}{2^{i-1}} \cdot \mathbb{G}_p^{\text{mul}}$ for the i^{th} round, $i = 1, \dots, \log_2 m$. In addition, updating $\hat{c}$ requires $1 \cdot \mathbb{Z}_p$. On the other hand, in Protocol.Col, the verifier updates $\hat{c}$ only, which involves $1 \cdot \mathbb{Z}_p$ for $\log_2 n$ round. Hence, for these subroutines, the verifier's computation cost becomes

$$(2m - 2) \cdot \mathbb{G}_p^{\text{exp}} + (2m - 2) \cdot \mathbb{G}_p^{\text{mul}} + (\log_2 m + \log_2 n) \cdot \mathbb{Z}_p$$

For AggMEC, the verifier needs to compute the aggregated commitments, namely, $A^{(i)}$, V , and P . Likewise the prover's AggMEC cost analyses, all these processes involve $(8 \log_2 n + 4 \log_2 m + 4) \cdot \mathbb{G}_t^{\text{exp}}$. In addition, the aggregation of all the `LeopardPC.Eval` requires a random linear combination of all the commitments being aggregated, which is $(3M + 2d_t + 3) \cdot \mathbb{G}_t$. After these processes, running `LeopardPC.Eval` of length D as a verifier's role suffices, which takes $\sqrt{D}$ operations in terms of $\mathbb{G}_p^{\text{exp}}, \mathbb{G}_2^{\text{exp}}$, and $2 \log_2 D \cdot \mathbb{G}_t^{\text{exp}}$. Hence, we can

summarize the total number of operations of the verifier in AggMEC as

$$\sqrt{D} \cdot \mathbb{G}_1^{\text{exp}} + \sqrt{D} \cdot \mathbb{G}_2^{\text{exp}} + (2 \log_2 D + 8 \log_2 n + 4 \log_2 m + 3M + 2d_t + 7) \cdot \mathbb{G}_t^{\text{exp}}.$$

We remark that, with the lazy MSM technique in Section B-C, the verifier can directly handle L_k, R_k, P_k as MSM bases instead of sequentially computing $A^{(i)}, V$. This idea enables avoiding some unnecessary computations over $\mathbb{G}_t$, and thus making $(6 \log_2 n + 2 \log_2 m) \cdot \mathbb{G}_t^{\text{exp}}$ operations sufficient instead of $(8 \log_2 n + 4 \log_2 m) \cdot \mathbb{G}_t^{\text{exp}}$.

D. PROOF SIZE

Since the concrete proof size has already been provided in Section V, we focus on the communications during AggMEC. In AggMEC, the communication involves sending $Q^{(i)} \in \mathbb{G}_t$ and $s, t^{(i)}, r^{(i)} \in \mathbb{Z}_p$, along with $\text{Plonkish}_{\text{Eval}}$ with the batched $\text{Leopard}_{\text{PCS.Eval}}$. For $\text{Plonkish}_{\text{Eval}}$, the prover needs to send pre-computed commitments C_{ID}, C_{r_i} for $i \in [M]$ for permutation consistency check, and commitments for the polynomials $Z, \{T_i\}_{i=1}^{d_t}, \{Q_i\}_{i=1}^{d_t-1}$ that represent constraint equations. Finally, the aggregation of $\text{Leopard}_{\text{PCS.Eval}}$ requires one extra communication to send the commitment of $\hat{q}(X)$, and running the final $\text{Leopard}_{\text{PCS.Eval}}$ involves sending $2 \log_2 D$ elements in $\mathbb{G}_t$.

To sum up, the overall communication cost for AggMEC is $(2 \log_2 D + 2M + 2d_t + 3) \cdot |\mathbb{G}_t|$, which coincides with the asymptotic complexity $O(\log_2 D)$ in Section V.



HYEONBUM LEE is a postdoctoral researcher at Seoul National University, Seoul, Republic of Korea. He received his Ph.D. degree from the Department of Mathematics at Hanyang University in 2025. He was a visiting researcher in the Department of Computer Science and Engineering at the University of Notre Dame from 2022 to 2023. His research interests include cryptography and computational theory.



SEUNGHUN PAIK received the B.S. degree in mathematics from Hanyang University, Seoul, Republic of Korea, in 2023. Since September 2023, he has been pursuing the Ph.D. degree in applied mathematics at the same university. From 2024 to 2025, he was a visiting researcher at the University of Notre Dame. His research interests include applied cryptography, privacy-preserving machine learning, and biometric template protection.



HYUNJUNG SON received the B.S. degree in mathematics from Sookmyung Women's University, Seoul, Republic of Korea, in 2023. Since March 2023, she has been pursuing the Ph.D. degree in applied mathematics at Hanyang University, Seoul, Republic of Korea. Her research interests include cryptography, especially zero-knowledge proof and private set operations.



JAE HONG SEO is a professor of Department of Mathematics at Hanyang University. He received the B.S. degree in mathematics from Korea University in 2004 and the Ph.D. degree from the Department of Mathematical Science, Seoul National University in 2011. From 2011 to 2013, he was with National Institute of Communications and Technology, Japan. From 2013 to 2018, he was with Myongji University, Korea. Since 2018, he has been an Associate Professor with the Department of Mathematics, Hanyang University, Korea. His research interests include computational algorithms, cryptology, and their applications to deep learning and blockchain.